IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN XXXX-XXXX, Vol. 0, No. 0, pp. 1–29. DOI:XXXXXXXX

Combined Indistinguishability Analysis

Verifying random probing leakage under random faults

Armand Schinkel and Pascal Sasdrich

Ruhr University Bochum, Bochum, Germany
{firstname.lastname}@rub.de

Abstract. Cryptographic hardware implementations are vulnerable to combined physical implementation attacks, integrating Side-Channel Analysis (SCA) and Fault Injection Analysis (FIA) to compromise their security. Although theoretically sound countermeasures exist, their practical application is often complicated and error-prone, making automated security verification a necessity. Various tools have been developed to address this need, using different approaches to formally verify security, but they are limited in their ability to analyze complex hardware circuits in the context of Combined Analysis (CA) and advanced probabilistic adversary models.

In this work, we introduce a novel verification method that assesses the security of complex hardware circuits in the context of *random probing with random faults*, a scenario that more closely reflects real-world combined attack scenarios. Our approach centers around symbolic fault simulation and the derivation of a *fault-enhanced leakage function* using the Fourier-Hadamard Transform (FHT), enabling the computation of tight leakage probabilities for arbitrary circuits and providing a more accurate and comprehensive security analysis. By integrating our method into the INDIANA security verification framework, we extended its capabilities to analyze the leakage behavior of circuits in the presence of random faults, demonstrating the practicality of our approach.

The results of our evaluation highlight the versatility and scalability of our approach, which can efficiently compute leakage probabilities under various fault scenarios for large-scale attacks, e. g., for a masked round of the PRESENT cipher. Notably, our method can complete most experiments in less than an hour, demonstrating a significant improvement over existing estimation-based tools. This achievement confirms the potential of our approach to provide a more comprehensive and practically useful security assessment of hardware circuits, and marks an important step forward for the development of secure hardware systems.

Keywords: Side-Channel Analysis · Fault Injection Analysis · Combined Analysis · formal verification · leakage function · Fourier-Hadamard Transform

1 Introduction

Physical Implementation Attacks. Physical implementation attacks exploit weaknesses of implementations that persist despite rigorous theoretical analysis of cryptographic algorithms, underscoring that a cryptographic algorithm is only as secure as its implementation. In particular, passive Side-Channel Analysis (SCA) leverages physical characteristics of electronic devices, such as runtime behavior [Koc96], power consumption [KJJ99], or electromagnetic (EM) emanations [GMO01], to exploit leakage of sensitive information. Among the numerous proposed countermeasures, *masking* is the predominant candidate due to its sound and information-theoretically secure foundations [CJRR99]. Beyond passive information leakage, active tampering poses another significant risk due to physical

implementation attacks. Fault Injection Analysis (FIA) is a well-known example, where adversaries use techniques such as clock glitches [DEG+18], voltage glitches [ZDCT13], EM pulses [DDRT12, DLM19], or focused laser beams [SA03] to disrupt correct cryptographic operations and reveal sensitive data. Modern countermeasures against fault injections typically rely on *redundancy* in time, area, or information to detect or correct transient faults.

Combined Implementation Security. Combined implementation security is an emerging field that addresses the domains of SCA and FIA no longer in isolation. In fact, the impact of combining active FIA and passive SCA has long been overlooked, but practical attacks have highlighted the urgency of this issue [AVFM07, CFGR10, RLK11, SJB+18, SBJ+21, SRJB23]. While early research attempted to integrate existing countermeasures for both SCA and FIA, these efforts often proved effective only in isolation, failing to protect against both passive and active attacks simultaneously [SMG16, RG20, GPK+21, DOT24]. Moreover, simply combining masking and redundancy techniques was shown to be insufficient to counteract all reciprocal effects [RLK11, APZ21, SBJ+21, RBFSG22, FGM+23, SRJB23]. Subsequent investigations have explored various approaches to build robust countermeasures against combined attacks, including MPC-based schemes [AIS18], MAC-based schemes [DAN+18], replication-based schemes [DN20, GPK+21, FRBSG22, FGM+23, PBGS24], and polynomial masking [BEF+23, ABEO24, FOWZ25]. However, designing and implementing secure cryptographic algorithms remains a complex manual process requiring expertise in hardware design, hardware security, and Electronic Design Automation (EDA). This challenge is exacerbated by the increasing complexity of modern designs, making formal verification a fundamental part of Very Large Scale Integration (VLSI) design cycles.

Formal Security Verification. Formal hardware verification, traditionally focused on functional correctness, is increasingly shifting towards implementation security, driven by hardware security researchers interested in formal models for active and passive adversaries and physical execution environments. For this, sound and accurate models can simplify the verification of functional correctness and security of cryptographic implementations, thereby shortening and streamlining development cycles. For masking, the Ishai-Sahai-Wagner (ISW) d-threshold probing model [ISW03] is commonly used, capturing basic information leakage through d adversarial probes that access intermediate values, although it falls short in covering more advanced attacks. The noisy-leakage model [PR13], while closer to practice, is less convenient for security proofs. Through a chain of security reductions, the p-random probing model [DDF14] serves as an intermediate, offering a more realistic approach than the d-threshold model and greater convenience for security proofs than the noisy-leakage model. However, it still lacks the ability to capture low-level physical effects due to simplified assumptions on independent but identical leakage probabilities. As a result, recent generalizations consider individual and independent leakage probabilities to better reflect uncovered physical effects [BFG+24]. In the field of FIA, formalization is less developed compared to SCA, without widely accepted standard fault models. The k-fault model [IPSW06] and the ζ-injection model [RSG23, TNN24], which are active counterparts to d-threshold probing, allow adversaries to inject a limited number of faults but do not fully capture practical scenarios. Recent efforts have led to the proposal of a random fault model [DN24, CDM+25] and a general random combined adversary model [BFG+24], aiming to address these gaps.

Mechanized Security Proofs. The increasing complexity of modern cryptographic designs has made manual security verification nearly infeasible, leading to the development of automated security reasoning and verification tools that enable mechanized security proofs. For passive information leakage, these tools are based on various principles, including veri-

fication of specific countermeasures [ANR18], direct security reasoning [BGI+18, BBC+19, BCP+20, KSM20, GHP+21, CFOS21, HB21, RBFSG22, MM22, BMRT22, HCP+24, BFG+25], or compositional reasoning [BGR18, BDM+20, CGLS21]. However, some tools may be incomplete due to the use of simulation-based techniques [MM22, HCP+24], allowing for false positives or negatives in security statements [BBC+19, BCP+20], or suffering from high computational complexity [KSM20, RBFSG22]. These tools can also be distinguished by their support for different models, such as the d-threshold probing [BGI+18, BBC+19, KSM20, GHP+21, HB21, RBFSG22, MM22], the p-random probing [BCP+20, CFOS21, BMRT22, BFG+25], and the noisy-leakage model [HCP+24]. In the context of active information tampering, verification tools evaluate specific countermeasures [AWMN20], employ SAT solvers [NOV+22, TGS+24], or use symbolic simulations [RBSS+21] to ensure the robustness of redundancy-based countermeasures. A review of prior work reveals that most tools are designed to address either SCA or FIA in isolation, with limited attention paid to combined adversary models. VERICA [RBFSG22] is a notable exception, but its scope is limited to the threshold probing and faulting models while suffering from computational complexity. Its recent extension, VERICA+ [BFG+24][1], has been proposed to implement verification in the general random combined model, however, it amplifies the issue of high computational complexity while only estimating bounds for random probing and random faulting adversaries. Notably, the recent introduction of INDIANA [BFG+25] has improved state-of-the-art security verification in the random probing model, using *indistinguishability analysis* and the Fourier-Hadamard Transform (FHT) for exact verification on arbitrary circuits. Nevertheless, the challenge of verifying security in the random combined adversary model, which combines the strengths of random probing and random faulting attacks, remains an open research question. This work aims to fill this gap by developing a framework for exact security verification in this more comprehensive and realistic combined setting, allowing to quantify (random) probing leakage in the presence of (random) faults.

Contributions. In this work, we present a comprehensive framework for verifying the security of masked digital hardware circuits against combined probing and faulting attacks, allowing to precisely and efficiently quantify random probing leakage under random faults. Our core verification concept is based on the derivation of *fault-enhanced leakage functions*, which takes into account the impact of faults on the circuit behavior in order to reason about the security of a circuit against random combined attacks. For this, our approach leverages symbolic fault simulation, enabling parallel analysis of faults on a large scale. This capability allows us to generate tight leakage bounds in the random probing model with random faults for complex circuits, whereas VERICA+ [BFG+24] is limited to estimating leakage probabilities due to constraints on the number of probes and faults. In addition to this, we also present a practical tool[2]that can efficiently compute the leakage probability of a circuit under combined attacks, while considering various adversarial scenarios, fault types, and fault distributions. Among others, the tool is demonstrated to be effective in evaluating the random probing security in the presence of random faults for various case studies, and provides the valuable insight that *unknown* faults do not increase the advantage of an adversary to impair the security of these gadgets.

Outline. Section 2 introduces preliminaries, including notation, circuit and security concepts, while Section 3 presents our concept to enable (random) probing leakage verification in the presence of (random) faults. Section 4 details key aspects of our automated reasoning techniques to support rapid, efficient, and systematic analysis of hardware circuits, while

[1]This work also presents ironMask+ which, however, is strongly limiting in its applicability as a universal Combined Analysis (CA) verification tool due to its inability to verify arbitrary circuit structures.

[2]The code is publicly available at github.

Table 1: Notation convention used throughout this paper.

	Notation	Description
Circuit	n_i	Number of unshared inputs to a circuit
	n_r	Number of randomness inputs to a (masked) circuit
	n_o	Number of unshared outputs of a circuit
	C	Circuit (original)
	$\mathsf{C}^{\mathcal{F}}$	Circuit (fault-enhanced)
	$\mathcal{W}$	Set of wires
	$\mathcal{G}$	Set of gates
Combined Analysis	s	Number of shares used by a masking scheme
	f	Number of injected faults
	d	Probing security order
	k	Fault security order
	$\mathcal{P}$	Set of probes
	$\mathcal{P}_i$	Probe on wire w_i
	$\mathcal{F}$	Set of faults
	$\mathcal{F}_i$	Fault on gate g_i
	τ	Type of injected fault
	A_{C}	Access function to circuit C
	$\mathcal{D}$	Wire leakage probability distribution
	p_i	Leakage probability $\in \mathcal{D}$
	$\mathcal{K}$	Fault injection probability distribution
	q_i	Fault probability $\in \mathcal{K}$

Section 5 demonstrates the effectiveness of our approach through evaluation of selected case studies. Finally, we conclude our work in Section 6.

2 Preliminaries

In this section, we establish the theoretical background for our work by introducing key concepts, definitions, notations, and our fundamental hardware circuit model, as well as the principles of *Boolean masking*, to create a solid foundation for understanding our subsequent contributions.

2.1 Notation

Throughout this work, we use a consistent notation convention, where functions are denoted in sans serif font (e. g., f), vectored functions are represented with uppercase characters (e. g., F), and sets are denoted with uppercase characters in calligraphic font (e. g., $\mathcal{S}$). A summary of specific notations is provided in Table 1.

2.2 Circuit Model

For this work, we model a deterministic digital logic circuit as Direct Acyclic Graph (DAG) $\mathsf{C} = (\mathcal{G}, \mathcal{W})$ where each node $g \in \mathcal{G}$ represents either a gate, an input, or an output and each edge $w \in \mathcal{W}$ represents a wire that connects two gates. Without loss of generality, we define combinational gates as $\mathcal{G}_{\mathsf{c}} = \{\mathsf{inv}, \mathsf{and}, \mathsf{nand}, \mathsf{or}, \mathsf{nor}, \mathsf{xor}, \mathsf{xnor}, \mathsf{mux}\}$. Further, we define sequential gates as $\mathcal{G}_{\mathsf{m}} = \{\mathsf{reg}\}$ which model clocked registers. With that, let the set of all gates $\mathcal{G}$ be $\mathcal{G}_{\mathsf{c}} \cup \mathcal{G}_{\mathsf{m}}$.

Definition 1 (Circuit)**.** A circuit C with n_i inputs and n_o outputs is a DAG $(\mathcal{G}, \mathcal{W})$, where each input node takes a label from $\{x_1, \ldots, x_{n_i}\}$, each output node takes a label from $\{y_1, \ldots, y_{n_o}\}$, and all other nodes take a label from $\mathcal{G}$. Furthermore, C has the following properties:

1. For each $u \in 1, ..., n_i$ there exists exactly one vertex labeled x_u representing an input, and for each $v \in 1, ..., n_o$ there exists exactly one vertex labeled y_v representing an output.
2. Each input vertex with label x_u has in-degree 0 and each output vertex with label y_v has in-degree 1 and out-degree 0.
3. Each vertex with a label in $\{\mathsf{and}, \mathsf{nand}, \mathsf{or}, \mathsf{nor}, \mathsf{xor}, \mathsf{xnor}\}$ has in-degree 2.
4. Each vertex with a label in $\{\mathsf{inv}, \mathsf{reg}\}$ has in-degree 1.
5. Each vertex with the label mux has in-degree 3.

A deterministic circuit C can be described functionally as a vectored Boolean function $\mathsf{F_C}\colon \mathbb{F}_2^{n_i} \to \mathbb{F}_2^{n_o}$ with the coordinate functions $\mathsf{f}_1^{\mathsf{C}}, ..., \mathsf{f}_{n_o}^{\mathsf{C}}$. Each wire $w \in \mathcal{W}$ represents a function $\mathsf{f}_w^{\mathsf{C}}\colon \mathbb{F}_2^{n_i} \to \mathbb{F}_2$ defined by the subgraph C_w with leaf w. Further, each gate $g \in \mathcal{G}$ (spanning a sub-circuit) computes a function $\mathsf{f}_g^{\mathsf{C}}\colon \mathbb{F}_2^{n_i} \to \mathbb{F}_2$ which is equal to the wire function $\mathsf{f}_w^{\mathsf{C}}$ where w is the output wire of g.

2.3 Boolean Masking

Addressing the threat of SCA, *Boolean masking* is most commonly used as it has sound and well studied theoretical foundations. With Boolean masking, each secret $x_i \in \mathbb{F}_2$ is split into s shares $\mathsf{Sh}(x_i) = \langle x_{i,0}, x_{i,1}, ..., x_{i,s-1} \rangle$, where each subset $\mathcal{X} \subset \mathsf{Sh}(x_i)$ is independent of $x_i = \bigoplus_{j=0}^{s-1} x_{i,j}$, i. e., computing x_i requires knowledge of all of its shares $x_{i,j}$ [CJRR99]. Any circuit $\mathsf{C}\colon \mathbb{F}_2^{n_i} \to \mathbb{F}_2^{n_o}$ can be transformed into a shared circuit $\mathsf{C}'\colon \mathbb{F}_2^{n_i \cdot s} \times \mathbb{F}_2^{n_r} \to \mathbb{F}_2^{n_o \cdot s}$ that operates on these shared values.

- $\mathsf{Enc}\colon \mathbb{F}_2^{n_i} \times \mathbb{F}_2^{(s-1)n_i} \to \mathbb{F}_2^{n_i \cdot s}$: The encoding function takes the n_i original unshared inputs $x_1, ..., x_{n_i}$, random bits $r_0 \in_R \mathbb{F}_2^{(s-1)n_i}$ and produces a valid sharing of the unshared input.
- $\mathsf{C}'\colon \mathbb{F}_2^{n_i \cdot s} \times \mathbb{F}_2^{n_r} \to \mathbb{F}_2^{n_o \cdot s}$: The masked version of the original circuit takes the shared input values $\mathsf{Sh}(x_1), ..., \mathsf{Sh}(x_{n_i})$ and random bits $r_1 \in_R \mathbb{F}_2^{n_r}$ to compute a valid output sharing. Formally, this means that $\forall x_i \in \mathbb{F}_2^{n_i}, y_i \in \mathbb{F}_2^{n_o}$ where $\mathsf{C}(x_i) = y_i$ it holds that $\mathsf{C}'(\mathsf{Sh}(x_i)) = \mathsf{Sh}(y_i)$.
- $\mathsf{Dec}\colon \mathbb{F}_2^{n_o \cdot s} \to \mathbb{F}_2^{n_o}$: The decoding function takes all valid output sharings $\mathsf{Sh}(y_i)$ and computes the unshared output values $y_i = \bigoplus_{j=0}^{s-1} y_{i,j}$.

The masked circuit C' is correct, if for all $x \in \mathbb{F}_2^{n_i}, r_0 \in \mathbb{F}_2^{(s-1)n_i}, r_1 \in \mathbb{F}_2^{n_r}$ it holds that:

$$\mathsf{C}(x) = \mathsf{Dec}(\mathsf{C}'(\mathsf{Enc}(x, r_0), r_1)).$$

3 Verification Concept

This section presents our new concept to verify the robustness of hardware circuits against information leakage in the presence of adversarial faults. We start by summarizing the adversary models that we consider, and then provide a detailed overview of our verification technique, which is designed to rapidly assess the resilience of arbitrary and complex hardware circuits in the context of (random) probing adversaries and the presence of (random) faults.

3.1 Adversary Models

We start by recalling key adversary models from the literature that deal with *passive* information leakage (SCA) and *active* information tampering (FIA). Based on this, we then

outline our considered *combined adversary model* for the characterization of information leakage in the presence of (adversarial) faults.

3.1.1 Passive Information Leakage

A large number of modern works on the verification of security properties in the presence of side-channel attackers are based on the fundamental *threshold d-probing model* presented by Ishai, Sahai, and Wagner [ISW03].

Threshold Probing Model. In the *threshold d-probing model*, an attacker is allowed to probe and learn a limited number of at most d wires of a circuit C, which can be arbitrarily chosen before operating the circuit. Based on these adversarial capabilities, the security of probing with respect to the *confidentiality* of the processed secrets is guaranteed if, for all combinations of at most d probes, the adversarial access to the circuit does not provide any information about the processed secrets. In other words, the distribution of the observed values is independent of the secrets and can be simulated without having access to any signal of the circuit.

The traditional threshold d-probing model has limitations in capturing real-world observations, as it neglects physical effects that can lead to additional information leakage. Specifically, *combinational recombinations (glitches)*, *memory recombinations (transitions)*, and *routing recombinations (couplings)* are well-documented sources of leakage that can compromise the security of a circuit. To address this, the *robust threshold probing model* [FGM+18] has been introduced, which extends the original probing concept to account for these physical effects. In this framework, an adversarial probe is enhanced to capture not only the probed signal but also additional sources that contribute to the information leakage. For example, a glitch-extended probe reveals the probed signal and all relevant stable signals, taking into account timing differences in signal propagation and processing, while assuming a worst-case glitch function that an adversary may observe. While similar extensions have been proposed for transitions and couplings, this work focuses on the glitch-extended threshold d-probing model, as glitches are a well-known primary source of unintentional physical leakage in modern circuits [MS06].

Random Probing Model. Although the robust threshold probing model provides a more realistic representation of real-world effects by accounting for physical phenomena, it still has limitations due to its assumption of precise and isolated leakage sources. In reality, observations are often noisy and aggregate information from multiple leakage sources, making it challenging to accurately model the leakage. The *random probing model* [DDF14] addresses these limitations by introducing a probabilistic approach, where an adversarial observation of a wire value succeeds with probability p and fails (leaking no information) with probability $1 - p$. Initially proposed as an auxiliary model to establish a reduction between the *threshold probing model* and the *noisy leakage model* [DDF14, PR13], the random probing model has gained significance as a standalone security model, offering a more realistic framework for security verification.

Recently, the original random probing model has been further generalized to allow for more accurate modeling of complex circuit and noise effects [BFG+24]. In this generalized version, the leakage probability of each wire is not restricted to a uniform and independent probability, but can instead follow an arbitrary distribution. This extension enables a more nuanced representation of advanced low-level circuit characteristics and noise patterns, making the model even more relevant for real-world security analyses.

However, while some research suggests that the random probing model can be combined with robust probe extensions to account for physical effects such as glitches and transitions [BMRT22], the exact approach for doing so is not yet well-established. To maintain a

clear and well-defined adversary model, we therefore focus on the non-robust (generalized) random probing model throughout this work [BFG+24].

3.1.2 Active Information Tampering

In contrast to the well-established and sophisticated models for passive information leakage, the development of formal adversary models for *active information tampering* has been slower to mature. However, recent years have seen a notable increase in research activity in this area, with new and improved fault adversary models being proposed with increasing frequency.

Threshold Faulting Model. The *threshold faulting adversary model*, inspired by the concepts of threshold probing adversaries, assumes an attacker that can precisely inject faults into at most k gates of a circuit C during its operation [IPSW06]. The adversarial capabilities can be further refined through a set of extended parameters, which define various aspects of the fault injection, such as the targeted locations and gates, the timing and duration of the faults, the size of the faults (in terms of the number of affected bits), and the type of faults introduced [RSG23, TNN24]. The integrity of the circuit operation is ensured if either the injected faults are detected or the faulty output is indistinguishable from the correct output, thereby preventing the adversary from exploiting the faults to compromise the security of the circuit.

Random Faulting Model. Recently, more sophisticated faulting models have emerged, building on the concept of random faults that occur with a certain probability q, while no fault effect is observable otherwise [DN23]. These models introduce an element of unpredictability, allowing for a more realistic representation of fault attacks. The idea of random faults has been further generalized by more advanced models, which assume that fault injections do not occur independently, but instead follow a specific distribution [BFG+24, CDM+25]. This extension enables the modeling of more complex fault scenarios, where the occurrence of faults is influenced by various factors, such as the hardware architecture, the injection mechanism, or the adversarial capabilities.

3.1.3 Information Leakage with Information Tampering

This work particularly focuses on the critical issue of information leakage in the presence of information tampering, where an adversary intentionally induces faults to compromise the security of a hardware circuit. Our focus on this topic stems from the observation that active tampering can significantly intensify information leakage, thereby reducing the overall security of the circuit. Conversely, we note that information leakage does not have a direct impact on the resilience of the circuit to fault injection attacks, making it a less critical aspect to study in the context of combined probing and faulting attacks.

To systematically analyze the interplay between information leakage and tampering, we consider the intersection of probing and faulting adversary models. This leads us to identify four distinct scenarios, each representing a unique combination of probing and faulting strategies:

Scenario 1: Threshold Probing with Threshold Faulting. In this scenario, we consider a combined adversary that is bounded by two thresholds: d for probing and k for faulting. Specifically, this scenario is characterized by a (d, k)-*combined adversary*, where the adversary can extract information from at most d wires and induce faults into at most k gates.

Scenario 2: Threshold Probing with Random Faulting. In this scenario, we consider a combined adversary that employs a threshold probing strategy to extract information,

while inducing faults randomly. Specifically, the adversary is limited to observing at most d wires, but has the ability to inject faults according to a distribution $\mathcal{K}$. This scenario is characterized by a $(d, \mathcal{K})$-*combined adversary*, where the threshold d bounds the adversarial observation, and the distribution $\mathcal{K}$ governs the random injection of faults.

Scenario 3: Random Probing with Threshold Faulting. In this scenario, we consider a combined adversary that uses a random probing strategy to extract information, while employing a threshold faulting approach to induce faults. The adversarial observations are governed by a distribution $\mathcal{D}$, which determines the likelihood of probing each wire. Meanwhile, the faulting attack is bounded by a threshold k, which limits the number of gates that can be affected by faults. This scenario is characterized by a $(\mathcal{D}, k)$-*combined adversary*, where the distribution $\mathcal{D}$ drives the random probing, and the threshold k constrains the faulting attack.

Scenario 4: Random Probing with Random Faulting. In this scenario, we consider a combined adversary that employs a random probing strategy to extract information, while also inducing faults randomly. The adversarial observations are driven by a distribution $\mathcal{D}$, which determines the likelihood of probing each wire, while the faults are governed by a separate distribution $\mathcal{K}$. This scenario is characterized by a $(\mathcal{D}, \mathcal{K})$-*combined adversary*, where both the probing and faulting attacks follow specific distributions.

3.2 Notions of Security

The definition of security varies across the four scenarios, as each scenario presents a unique combination of probing and faulting attacks. In general, an attack is considered successful, and a hardware circuit is deemed insecure, if the amount of leaked information exceeds a certain threshold. The specific threshold and the conditions for a successful attack differ depending on the scenario.

In the first two scenarios, which involve a threshold probing adversary, an attack is successful if the adversary can learn the processed secret using at most d probes. This means that the security of the circuit is compromised if the adversary can extract the secret information within the limited number of probes.

In contrast, the last two scenarios, which involve a random probing adversary, consider an attack successful if the leakage probability exceeds a certain threshold ϵ. This means that the security of the circuit is compromised if the probability of information leakage exceeds a security threshold ϵ, regardless of the number of probes used.

In the following paragraphs, we provide a formal definition of security for each of the four scenarios, which builds upon the related work in [BFG+24] but differs in our focus on combined leakage without considering security in the presence of faults only. However, it is worth noting that our definition of security can be easily extended to include the fault resilience of a circuit C, which would immediately yield a definition of security according to the *general random combined model* as defined in [BFG+24].

To provide a formal definition of security, we first introduce three key functions that enable us to model the behavior of a circuit under fault combinations, randomly select a probe combination, and compute the output values that an adversary would obtain when probing a specific wire for a given input.

`AssignFaultGates`$(\mathsf{C}, \mathcal{K})$: Given a circuit C, the *faulty-gate sampler* generates a fault combination $\mathcal{F}$ according to a probability distribution $\mathtt{Pr}_{\mathcal{K}}[\mathcal{F}]$, and produces a corresponding modified circuit $\mathsf{C}^{\mathcal{F}}$ denoted as the *fault-enhanced circuit*.

`LeakingWires`$(\mathsf{C}, \mathcal{D})$: Given a circuit C, the *leaking-wire sampler* selects a probe combination $\mathcal{P}$ according to a probability distribution $\mathtt{Pr}_{\mathcal{D}}[\mathcal{P}]$.

$\texttt{AssignWires}(\mathsf{C}, x, \mathcal{P})$: Given a circuit C and a fixed input x, the *assign-wire sampler* outputs the values assigned to the probes $p \in \mathcal{P}$ as a tuple in $\mathbb{F}_2^{|\mathcal{P}|}$.

The adversarial perspective on probing leakage in the presence of faults, whether random or not, can be formally represented by an observation function $\mathsf{O}_{\mathcal{D},\mathcal{K}}(\mathsf{C}, x)$. This function is defined through an experiment that captures the interaction between the adversarial probing strategy, the circuit behavior, and the effects of faults, providing a mathematical framework for analyzing the leakage of sensitive information in the presence of faults.

$$\begin{aligned} \mathsf{C}^{\mathcal{F}} &\leftarrow \texttt{AssignFaultGates}(\mathsf{C}, \mathcal{K}) \\ \mathcal{P} &\leftarrow \texttt{LeakingWires}(\mathsf{C}^{\mathcal{F}}, \mathcal{D}) \\ \mathsf{O}_{\mathcal{D},\mathcal{K}}(\mathsf{C}, x) &\leftarrow \texttt{AssignWires}(\mathsf{C}^{\mathcal{F}}, x, \mathcal{P}) \end{aligned}$$

Several key points are worth noting at this stage. Firstly, although the experiment described is specifically tailored to the random probing and random faulting scenarios, where probes and faults follow distributions $\mathcal{D}$ and $\mathcal{K}$, it can be easily adapted to accommodate the threshold probing and threshold faulting scenarios. By carefully selecting the distributions $\mathcal{D}$ and $\mathcal{K}$, we can effectively model the situation where an adversary is limited to placing at most d probes and injecting at most k faults. Additionally, it is important to highlight that our leakage function focuses solely on the probing leakage in fault-enhanced circuits, without considering the security degradations caused by faults only. This is a deliberate design choice, driven by the observation that probing does not impact fault security (as already explained above), driving us to solely concentrate on the combined probing with faults scenario. Consequently, our experiment differs from the definitions and experiments presented in [BFG+24], which take into account both probing and fault security. However, if we were to extend our experiment to include fault security, we would immediately yield the general random combined security definition, as described in [BFG+24].

Definition 2 (General Random Probing Security with Random Faults)**.** A circuit C is considered to be $(\mathcal{D}, \mathcal{K}, \epsilon)$-*random combined secure* if there exists a simulator $\texttt{Sim}$ such that for all possible inputs x, the statistical distance between the distributions of the simulator $\texttt{Sim}$ and the observed leakage function is upper-bounded by ϵ, i. e., the two distributions are ϵ-close:

$$\texttt{Sim}(\mathsf{C}, \mathcal{P}) \backsimeq_{\epsilon} \mathsf{O}_{\mathcal{D},\mathcal{K}}(\mathsf{C}, x).$$

Our verification approach centers around the exact and efficient derivation of a *leakage function*, which determines whether the observed distribution can be distinguished from a perfectly simulated distribution. This enables the security evaluation of the circuit for any given probe and fault distribution, as long as the input distribution is known, by assessing whether the observed leakage is indistinguishable from a random distribution.

3.3 Verification Approach

In this work, we model the leakage function as a Boolean function that maps all possible combinations of probes and faults to an outcome, indicating whether the combination reveals sensitive information (*true*) or not (*false*), i. e., the observed (probed) distribution is distinguishable from the perfect simulation or not. In this section, we delve into the details of how to construct, retrieve, and utilize this leakage function in the context of the four scenarios identified earlier, providing a comprehensive framework for verifying the security of circuits against probing attacks in the presence of faults.

3.3.1 Passive Information Leakage

Our verification approach for passive information leakage significantly extends the methodology introduced by Beierle et al. [BFG+25], which we outline below for completeness.

Adversarial Access to a Circuit. The (unlimited) perspective of a probing adversary can be formally captured by the concept of *access to a circuit*. This access refers to ability of the adversary to observe and probe the circuit internals. Formally, the access can be defined as follows:

Definition 3 (Access to a Circuit)**.** For a circuit $\mathsf{C} = (\mathcal{G}, \mathcal{W})$ implementing a function $\mathsf{C}\colon \mathbb{F}_2^{n_i} \to \mathbb{F}_2^{n_o}$, the access to that circuit $\mathsf{A}_\mathsf{C}\colon \mathbb{F}_2^{n_i} \to \mathbb{F}_2^{|\mathcal{W}|}$ yields a vector of values corresponding to each wire function instantiated with $x \in \mathbb{F}_2^{n_i}$. Each coordinate of A_C corresponds to a wire in C, therefore $\mathsf{A}_\mathsf{C}(x) = (\mathsf{f}^\mathsf{C}_{w_0}, ..., \mathsf{f}^\mathsf{C}_{w_{|\mathcal{W}|}})$ represents the state of the circuit C when the input is instantiated with x.

Specifically, the coordinates of the access function represent the values of all the wires in C, providing a complete snapshot of the circuit state for a given input.

Leakage Function. With such an access function and an encoding function (which corresponds to a specific masking scheme), we can define the leakage function of the circuit in the presence of a probing adversary. The leakage function, which captures the information that the adversary can potentially extract from the circuit, is defined as follows:

Definition 4 (Leakage Function)**.** Let $\mathsf{H} \coloneqq \mathsf{A}_\mathsf{C} \circ \mathsf{Enc}$ be a function from $\mathbb{F}_2^{n_i} \times \mathbb{F}_2^{(s-1)n_i+n_r}$ to $\mathbb{F}_2^{|\mathcal{W}|}$, given an encoding function $\mathsf{Enc}\colon \mathbb{F}_2^{n_i} \times \mathbb{F}_2^{(s-1)n_i} \to \mathbb{F}_2^{n_i \cdot s}$ and an access function $\mathsf{A}_\mathsf{C}\colon \mathbb{F}_2^{n_i \cdot s} \times \mathbb{F}_2^{n_r} \to \mathbb{F}_2^{|\mathcal{W}|}$ to a masked C for the function $\mathsf{F}_\mathsf{C}\colon \mathbb{F}_2^{n_i \cdot s} \times \mathbb{F}_2^{n_r} \to \mathbb{F}_2^{n_o \cdot s}$.

The leakage of the probe set $\mathcal{P}$ in the masked circuit C is defined as the probability distribution of the probe values, where the probabilities are computed over uniform random choices of the masking randomness $r \in \mathbb{F}_2^{(s-1)n_i+n_r}$:

$$\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_\mathsf{C})}\colon 2^{\mathcal{W}} \to \{0,1\}$$
$$\mathcal{P} \mapsto \begin{cases} 0, \text{if } \forall x \in \mathbb{F}_2^{n_i}, y \in \mathbb{F}_2^{|\mathcal{P}|}\colon \mathsf{Pr}_r[\mathsf{H}|_\mathcal{P}(x,r) = y] = \mathsf{Pr}_r[\mathsf{H}|_\mathcal{P}(0,r) = y] \\ 1, \text{else} \end{cases}$$

The derivation of the leakage function is based on the concept of *indistinguishability of secret-dependent adversarial observations*. In other words, the leakage function is constructed by analyzing whether an adversary can distinguish between the distributions of probed values for different (unmasked) secrets processed by the circuit. If the adversary is unable to distinguish between these distributions for any given secret, then the corresponding probe combination is considered non-leaking. On the other hand, if the adversary can distinguish between the distributions, it implies that the probe combination is leaking, and the adversary can potentially extract sensitive information from the circuit.

Verifying Threshold Probing Security. The verification of d-threshold probing security for a (masked) circuit can be performed by analyzing the leakage function as defined above. Specifically, we need to verify that the leakage function $\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_\mathsf{C})}(\mathcal{P}_i) = 0$ for all probe sets $\mathcal{P}_i$ with a Hamming weight of d or less, i. e., $\mathsf{HW}(\mathcal{P}_i) \leq d$. This check ensures that the circuit does not leak sensitive information when probed with at most d probes. By confirming that the leakage function satisfies this condition for all relevant probe sets, we can establish the security of the circuit against a d-threshold probing adversary.

Algorithm 1: Fault-enhancing circuit transformation.

Require: Circuit $\mathsf{C} = (\mathcal{G}, \mathcal{W})$, its functional description F_C, fault type τ.
Ensure: Functional description $\mathsf{F}_{\mathsf{C}^\mathcal{F}}$ of fault-enhanced circuit $\mathsf{C}^\mathcal{F} = (\mathcal{W}^\mathcal{F}, \mathcal{G}^\mathcal{F})$, set of fault select variables $\mathcal{X}_{sel}$.

$\mathcal{X}_{sel} \leftarrow \emptyset$
$\mathsf{F}_{\mathsf{C}^\mathcal{F}} \leftarrow \mathsf{F}_\mathsf{C}$
for $g \in \mathcal{G}$ **do**
  Let $x_g \in \mathbb{F}_2$
  $\mathcal{X}_{sel} \leftarrow \mathcal{X}_{sel} \cup x_g$
  if $\tau =$ `set` **then**
    $\mathsf{f}_g^{\mathsf{C}^\mathcal{F}} \leftarrow x_g \vee \neg x_g \wedge \mathsf{f}_g^{\mathsf{C}^\mathcal{F}}$
  else if $\tau =$ `reset` **then**
    $\mathsf{f}_g^{\mathsf{C}^\mathcal{F}} \leftarrow \neg x_g \wedge \mathsf{f}_g^{\mathsf{C}^\mathcal{F}}$
  else if $\tau =$ `bitflip` **then**
    $\mathsf{f}_g^{\mathsf{C}^\mathcal{F}} \leftarrow x_g \wedge \neg \mathsf{f}_g^{\mathsf{C}^\mathcal{F}} \vee \neg x_g \wedge \mathsf{f}_g^{\mathsf{C}^\mathcal{F}}$
  else
    $\mathsf{f}_g^{\mathsf{C}^\mathcal{F}} \leftarrow x_g \wedge \tau \vee \neg x_g \wedge \mathsf{f}_g^{\mathsf{C}^\mathcal{F}}$
return $\mathsf{F}_{\mathsf{C}^\mathcal{F}}, \mathcal{X}_{sel}$

Verifying Random Probing Security. In the (general) random probing model, verifying the security of a circuit involves calculating the overall leakage probability ϵ. This is done by accumulating the contributions of all possible probe sets that leak sensitive information, weighted by the likelihood p_i of each probe being leaked to the adversary. More formally, the leakage probability ϵ is computed as:

$$\epsilon = \sum_{i=0}^{|\mathcal{W}|} p_i \cdot \mathsf{L}_{(\mathsf{Enc},\mathsf{A}_\mathsf{C})}(\mathcal{P}_i).$$

3.3.2 Information Leakage with Information Tampering

Our verification approach takes a significant step forward by considering the impact of adversarial faults on information leakage. A crucial observation that underlies our method is that the concept of a leakage function can be naturally extended to account for faults, allowing us to rapidly evaluate the security of a circuit C in the face of combined adversaries (according to the four scenarios outlined above). The following paragraphs provide a brief outline of our extension to the leakage function concept, which enables us to consider the effects of both threshold and random fault adversaries on information leakage, and to verify the security of circuits against these combined adversaries.

Circuit Transformation. Our approach starts with a circuit transformation that incorporates fault locations and functions into the circuit, where each gate is associated with a binary activation variable x_{sel} that indicates the presence or absence of a fault (as described in Algorithm 1). This transformation allows us to model the effect of faults on the circuit behavior while maintaining the flexibility to retain the original circuit function and derive the faulty circuit function for a specific fault combination as needed.

In addition to this, we incorporate the fault model that defines how a fault impacts the targeted gate in the circuit if the fault activation variable x_{sel} is enabled. The considered fault models can range from simple effects such as `reset`, `set`, or `bitflip`, to more complex functions that depend on the inputs of the circuit or gates. In this regard, our approach is

highly flexible, allowing it to accommodate fault functions of arbitrary complexity, and thereby enabling the modeling of a broad spectrum of fault adversaries.

The resulting *fault-enhanced circuit*, denoted as $\mathsf{C}^{\mathcal{F}}$, implements a function $\mathsf{F}_{\mathsf{C}^{\mathcal{F}}} \colon \mathbb{F}_2^{n_i} \times \mathbb{F}^{|\mathcal{G}|} \mapsto \mathbb{F}_2^{n_o}$ according to the considered fault model while the access to a fault-enhanced circuit $\mathsf{C}^{\mathcal{F}}$ still only observes the wires in the modified circuit. In other words, the access to the fault-enhanced circuit is defined as a function $\mathbb{F}_2^{n_i} \times \mathbb{F}^{|\mathcal{G}|} \mapsto \mathbb{F}^{|\mathcal{W}|}$, where $\mathcal{W}$ is the set of wires in the original circuit, and exact knowledge of the fault activation variables are not provided to an adversary (i. e., we consider an *unknown-fault scenario* here).

Fault-Enhanced Leakage Function. Based on the circuit transformation, we extend the definition of the leakage function in order to yield the notion of a *fault-enhanced leakage function*. This enhanced leakage function then can be used to reason about the security of hardware circuits in the combined adversary scenarios identified above. Formally, the fault-enhanced leakage function is defined as follows:

Definition 5 (Fault-Enhanced Leakage Function)**.** Let $\mathsf{H}^{\mathcal{F}} \coloneqq \mathsf{A}_{\mathsf{C}^{\mathcal{F}}} \circ \mathsf{Enc}$ be a function from $\mathbb{F}_2^{n_i} \times \mathbb{F}_2^{(s-1)n_i+n_r} \times \mathbb{F}_2^{|\mathcal{G}|}$ to $\mathbb{F}_2^{|\mathcal{W}|}$, given an encoding function $\mathsf{Enc} \colon \mathbb{F}_2^{n_i} \times \mathbb{F}_2^{(s-1)n_i} \to \mathbb{F}_2^{n_i \cdot s}$ and an access function $\mathsf{A}_{\mathsf{C}^{\mathcal{F}}} \colon \mathbb{F}_2^{n_i \cdot s} \times \mathbb{F}_2^{n_r} \times \mathbb{F}_2^{|\mathcal{G}|} \to \mathbb{F}_2^{|\mathcal{W}|}$ to a fault-enhanced $\mathsf{C}^{\mathcal{F}}$ that implements the function $\mathsf{F}_{\mathsf{C}^{\mathcal{F}}} \colon \mathbb{F}_2^{n_i \cdot s} \times \mathbb{F}_2^{n_r} \times \mathbb{F}_2^{|\mathcal{G}|} \to \mathbb{F}_2^{n_o \cdot s}$.

The leakage of the probe set $\mathcal{P}$ in the fault-enhanced circuit $\mathsf{C}^{\mathcal{F}}$ under a fault set $\mathcal{F}$ is defined as the probability distribution of the probe values restricted to the fault values, where the probabilities are computed over uniform random choices of the masking randomness $r \in \mathbb{F}_2^{(s-1)n_i+n_r}$:

$$\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})} \colon 2^{\mathcal{W}} \times 2^{\mathcal{G}} \to \{0,1\}$$

$$\mathcal{P} \times \mathcal{F} \mapsto \begin{cases} 0, \text{if } \forall x \in \mathbb{F}_2^{n_i}, y \in \mathbb{F}_2^{|\mathcal{P}|} \colon \mathsf{Pr}_r[\mathsf{H}^{\mathcal{F}}|_{\mathcal{P}}(x,r) = y] = \mathsf{Pr}_r[\mathsf{H}^{\mathcal{F}}|_{\mathcal{P}}(0,r) = y] \\ 1, \text{else} \end{cases}$$

Verifying Probing Security with Faults. The verification of probing security in the presence of faults for the four combined adversaries outlined in Section 3.1.3 involves calculating a fault-enhanced leakage probability, ϵ_f, which accounts for the impact of random faults on the circuit security. The calculation of ϵ_f is based on the formula:

$$\epsilon_f = \sum_{j=0}^{|\mathcal{G}|} q_j \cdot \sum_{i=0}^{|\mathcal{W}|} p_i \cdot \mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}(\mathcal{P}_i, \mathcal{F}_j).$$

For threshold adversaries, the verification process restricts the probe set probabilities (p_i) and fault set probabilities (q_i) based on the Hamming weights of the probe sets and fault sets. Specifically, $p_i = 1$ if $\mathsf{HW}(\mathcal{P}_i) \leq d$, and $p_i = 0$ otherwise, while $q_i = 1$ if $\mathsf{HW}(\mathcal{F}_i) \leq k$, and $q_i = 0$ otherwise. By applying these restrictions, we can compute the fault-enhanced leakage probability ϵ_f, which provides a measure of the circuit's security against the considered combined adversary models. In particular, for the (d,k)-combined adversary, the value of $\epsilon_f = 0$ confirms security, indicating the ability of the circuit to withstand combined threshold attacks.

4 Automated Reasoning

In this section, we describe implementation details and explain how our framework verifies the security of circuits in the random probing model with random faults, as defined by the identified adversarial scenarios according to Section 3.1.3.

4.1 Fault-Enhanced Leakage Detection

As described by Beierle et *al.*, the leakage function $\mathsf{L}_{(\mathsf{Enc},\mathsf{A_C})}$ can be linked to the FHT over the difference of output distributions under fixed inputs [BFG+25].

More specifically, the FHT of a function $\mathsf{f}: \mathbb{F}_2^n \to \mathbb{Z}$ computes its decomposition in the spectral representation, i. e., f is expressed as sum of parity functions. Formally, the FHT $\widehat{\mathsf{f}}: \mathbb{F}_2^n \to \mathbb{Z}$ is defined as:

$$\widehat{\mathsf{f}}\colon \mathbb{F}_2^n \to \mathbb{Z}, \quad \beta \mapsto \sum_{y \in \mathbb{F}_2^n} (-1)^{\langle \beta, y \rangle} \mathsf{f}(y),$$

where $\langle \beta, y \rangle$ denotes the canonical inner product of the vectors $\beta, y \in \mathbb{F}_2^n$. Notably, the Fast Fourier-Hadamard Transform (FFHT) can be computed over a function $\mathsf{f}: \mathbb{F}_2^n \to \mathbb{Z}$ with a complexity of $O(n \log n)$ and can be computed efficiently when represented as Multi-Terminal Binary Decision Diagram (MTBDD).

4.1.1 Fourier-Hadamard Transform for Fault-Enhanced Leakage Detection.

As part of this work, we extend this link between leakage and FHT by computing the FHT over the encoded access to the fault-enhanced circuit. For any probe set $\mathcal{P} \subseteq \mathcal{W}$, we define $\mathtt{prec}(\mathcal{P}) = \{\beta \in \mathbb{F}_2^{|\mathcal{W}|} | \beta_i = 0 \text{ if } w_i \notin \mathcal{P}\}$ which holds bit masks for all subset combinations of $\mathcal{P}$. Further, we denote the preimages of $y \in \mathbb{F}_2^{n_o}$ under a function $\mathsf{F}_x : \mathbb{F}_2^{(s-1)n_i+n_r} \to \mathbb{F}_2^{n_o}$ as $(\mathsf{F})_x^{-1}(y) := \{r \in \mathbb{F}_2^{(s-1)n_i+n_r} | \mathsf{F}(x,r) = y\}$. $(\mathsf{H}^{\mathcal{F}})_x^{-1}$ therefore yields all $r \in \mathbb{F}_2^{(s-1)n_i+n_r}$ for which the wires of the fault-enhanced circuit $\mathsf{C}^{\mathcal{F}}$ evaluate to $y = (f_1(x), ..., f_{|\mathcal{W}|}(x))$.

Theorem 1. *Let* $\mathsf{Enc}\colon \mathbb{F}_2^{n_i} \times \mathbb{F}_2^{(s-1)n_i} \to \mathbb{F}_2^{n_i \cdot s}$ *be an encoding and* $\mathsf{A}_{\mathsf{C}^{\mathcal{F}}}\colon \mathbb{F}_2^{n_i \cdot s} \times \mathbb{F}_2^{n_r} \times \mathbb{F}_2^{|\mathcal{G}|} \to \mathbb{F}_2^{|\mathcal{W}|}$ *be the access to a masked fault-enhanced circuit* $\mathsf{C}^{\mathcal{F}}$ *implementing a function* $\mathsf{F_C}\colon \mathbb{F}_2^{n_i \cdot s} \times \mathbb{F}_2^{n_r} \to \mathbb{F}_2^{n_o \cdot s}$. *Further, let* $\mathsf{H}^{\mathcal{F}} = \mathsf{A}_{\mathsf{C}^{\mathcal{F}}} \circ \mathsf{Enc}$ *and* $\mathsf{D}_{\mathsf{H},x}(\beta) = |(\mathsf{H})_x^{-1}(\beta)| - |(\mathsf{H})_0^{-1}(\beta)|$. *Then,* $\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}(\mathcal{P},\mathcal{F}) = 0$ *iff for all* $x \in \mathbb{F}_2^{n_i}$, *it holds that:*

$$\forall \beta \in \mathtt{prec}(\mathcal{P}), \forall \gamma \in \mathtt{prec}(\mathcal{F})\colon \widehat{\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}|_\gamma}(\beta) = 0.$$

Please note that our Theorem 1 is a simple extension of Theorem 1 in [BFG+25], incorporating fault-enhanced leakage functions with fixed faults. To avoid redundancy, we will not present a formal proof but will instead offer an intuitive explanation of how the original proof can be adapted. In particular, the extension requires that the FHT of the frequency difference is zero not only for the probe set $\mathcal{P}$ and its subsets $\mathtt{prec}(\mathcal{P})$ in functions without fault-enhancement, but also for the fault set $\mathcal{F}$ and its subset $\mathtt{prec}(\mathcal{F})$. The rest of the proof then proceeds in consistency with the original proof.

With this parity function decomposition, we can identify the input, probe, and fault combinations that have the most significant impact on the frequency difference. This, in turn, reveals the specific probe combinations for which the secret-restricted frequencies are distinguishable. Utilizing the aforementioned theorem, we can compute the fault-enhanced leakage function over the probe sets $\mathcal{P}$ and fault sets $\mathcal{F}$ by aggregating the FFHT decompositions across all possible input, probe, and fault combinations:

$$\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}(\mathcal{P},\mathcal{F}) = \begin{cases} 0, \text{if } \sum_{x \in \mathbb{F}_2^{n_i}} \sum_{\gamma \in \mathtt{prec}(\mathcal{F})} \sum_{\beta \in \mathtt{prec}(\mathcal{P})} \widehat{\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}|_\gamma}(\beta) = 0 \\ 1, \text{else} \end{cases} \tag{1}$$

Notably, our verification approach uses symbolic fault simulation to enhance the circuit model with symbolic faults, rather than explicitly constructing each faulty circuit. Applying the FHT to this fault-enhanced model yields a parametrized leakage function (according to Theorem 1), which mitigates complexity by avoiding repeated application of the FHT

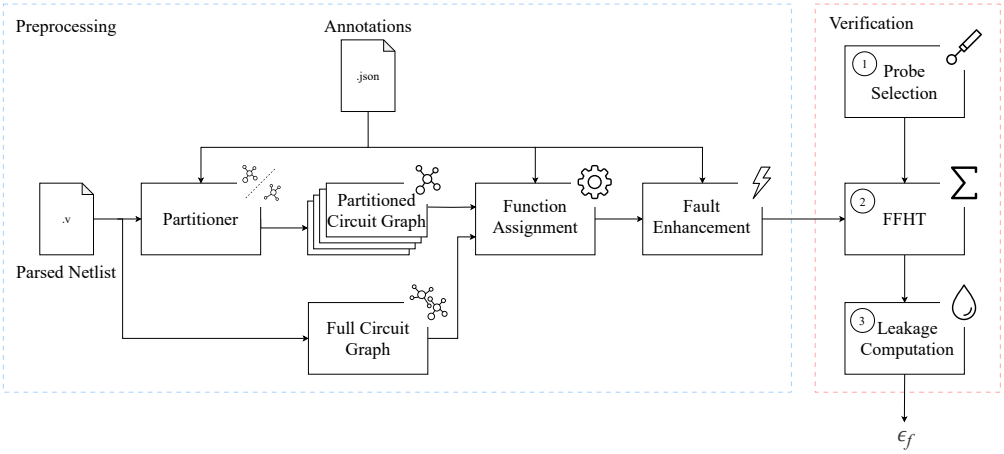

Figure 1: Overview of general verification flow with preprocessing on the left and security verification on the right. We provide details on the circuit partitioning in Section 4.2.2 and details on fault enhancement in Section 3.3.2.

for each faulty circuit. Although symbolic faults introduce exponential complexity to the circuit model, we partially counteract this by leveraging redundancies in the Binary Decision Diagram (BDD) and MTBDD data structures, significantly improving verification efficiency compared to iterative simulation of individual fault combinations.

4.2 Implementation

In this section, we now describe how to implement our fault-enhanced leakage detection approach using BDDs and MTBDDs to enable practical security reasoning about circuits in the random leakage model with random faults.

Assumptions and Constraints. We establish the following initial assumptions and constraints for our practical verification procedure. Our analysis focuses on non-iterative circuits that can be modeled as a DAG, which can be obtained by unrolling iterative circuits through preprocessing. Additionally, we require circuits to be annotated with their corresponding fault domains, input and output share domains, and share indices, which is common since our tool is targeted towards circuit designers who are familiar with their designs. We also assume independent and identically distributed inputs, including secrets $x \in \mathbb{F}_2^{n_i}$, initial sharings $r_0 \in \mathbb{F}_2^{(s-1)n_i}$, and refreshings $r_1 \in \mathbb{F}_2^{n_r}$. Note that these assumptions are specific to the implementation of our concept and do not impose general limitations on the approach.

4.2.1 Verification of Random Probing Security with Random Faults.

As discussed before, the ultimate goal of our automated security reasoning process is to retrieve the fault-enhanced leakage function $\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}(\mathcal{D},\mathcal{K})$, which can then be used to calculate the leakage probability ϵ_f for a given circuit, given a fault probability distribution $\mathcal{K}$ and a leakage probability distribution $\mathcal{D}$.

Verification Flow. The general verification flow consists of three steps: (1) selecting a probe set $\mathcal{P}$ and determining the output distribution of the probes for uniformly distributed secrets, (2) applying the FFHT to this output distribution, and (3) computing the leakage

Algorithm 2: FFHT over fault enhanced circuit frequency difference $\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}$ using MTBDD operations (adapted from [BFG+25]).

Require: Difference between adversary views of fault-enhanced circuit of all input combinations and a fixed input $\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x} : \mathbb{F}_2^{|\mathcal{W}|} \to \mathbb{Z}$.

Ensure: FHT $\widehat{\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}} : \mathbb{F}_2^n \to \mathbb{Z}$.

$\widehat{\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}} \leftarrow \mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}$
for $x_i \in x$ **do**
  $\mathsf{D}_0 \leftarrow \widehat{\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}}|_{x_i=0}$ ▷ RESTRICT
  $\mathsf{D}_1 \leftarrow \widehat{\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}}|_{x_i=1}$ ▷ RESTRICT
  $\widehat{\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}} \leftarrow \neg x_i(\mathsf{D}_0 + \mathsf{D}_1) + x_i(\mathsf{D}_0 - \mathsf{D}_1)$ ▷ ITE
return $\widehat{\mathsf{D}_{\mathsf{H}^{\mathcal{F}},x}}$

function from the resulting decomposition, as illustrated in Figure 1. All steps are described in more detail below, providing a comprehensive overview of the verification process.

Output Distribution at Probeset Locations. To determine the output distribution for a given probe set $\mathcal{P}$, we utilize a frequency transition function, as defined below, that calculates the probe set distribution based on a given input distribution.

Definition 6 (Frequency Transition)**.** Let $\mathsf{Enc}\colon \mathbb{F}_2^{n_i} \times \mathbb{F}_2^{(s-1)n_i} \to \mathbb{F}_2^{n_i \cdot s}$ be a circuit encoding that generates a valid sharing and $\mathsf{A}_{\mathsf{C}^{\mathcal{F}}}\colon \mathbb{F}_2^{n_i \cdot s} \times \mathbb{F}_2^{n_r} \times \mathbb{F}_2^{|\mathcal{G}|} \to \mathbb{F}_2^{|\mathcal{W}|}$ be the access to a masked, fault-enhanced circuit $\mathsf{C}^{\mathcal{F}}$. Further, let $\mathsf{H}^{\mathcal{F}} := \mathsf{A}_{\mathsf{C}^{\mathcal{F}}} \circ \mathsf{Enc}$ be the encoded access to $\mathsf{C}^{\mathcal{F}}$ with $\mathsf{H}^{\mathcal{F}}\colon \mathbb{F}_2^{n_i} \times \mathbb{F}_2^{(s-1)n_i+n_r} \times \mathbb{F}_2^{|\mathcal{G}|} \to \mathbb{F}_2^{|\mathcal{W}|}$. We then define the transition function $\mathsf{T}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}\colon \mathbb{F}_2^{s \cdot n_i} \times \mathbb{F}_2^{n_r} \times \mathbb{F}_2^{|\mathcal{P}|} \to \mathbb{F}_2$ as follows:

$$\mathsf{T}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}},\mathcal{P})}(x,r,y) = \begin{cases} 1, \text{if } \mathsf{H}^{\mathcal{F}}|_{\mathcal{P}}(x,r) = y \\ 0, \text{else} \end{cases}$$

Practically, given the access to the masked, fault-enhanced circuit for a given probe set $\mathsf{A}_{\mathsf{C}^{\mathcal{F}}}|_{\mathcal{P}}$, where $\mathsf{A}_{\mathsf{C}^{\mathcal{F}}}|_{\mathcal{P}_i}$ is the i-th coordinate function of this access and y_i is the i-th bit of the output y, the transition can be computed as:

$$\mathsf{T}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}},\mathcal{P})}(x,r,y) = \prod_{i=0}^{|\mathcal{P}|-1} \neg(\mathsf{H}^{\mathcal{F}}|_{\mathcal{P}_i}(x,r) \oplus y_i).$$

With this frequency transition, we can now compute the discrete output distribution at our probe locations $\mathcal{P}_i \in \mathcal{P}$ with:

$$|(\mathsf{H}^{\mathcal{F}}|_{\mathcal{P}})_x^{-1}(y)| = \exists_r(\mathsf{T}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}},\mathcal{P})}(x,r,y)).$$

Note that the existential quantification over r is performed in $\mathbb{R}$ and results in an output frequency at the probe locations in $\mathcal{P}$ that is independent of the initial and refreshing random bits $r = (r_0, r_1) \in \mathbb{F}_2^{(s-1)n_i} \times \mathbb{F}_2^{n_r}$, which are uniformly distributed. Additionally, scaling the transition relation according to the distribution of the inputs x allows to consider and compute different observations for secret-dependent input distributions. Accordingly, this procedure naturally maps and extends to the application of BDDs and MTBDDs.

Frequency Decomposition with the FFHT. The next step involves performing a frequency decomposition of $\mathsf{D}_{\mathsf{H}^{\mathcal{F}}}$ using the FFHT, as outlined in Algorithm 2. This process maps well to MTBDDs and their supported operations, allowing for efficient computation.

Algorithm 3: Computation of leakage probability ϵ_f with leakage function $\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}$.

Require: Leakage function $\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}$, leakage probability distribution $\mathcal{D}$, fault probability distribution $\mathcal{K}$, wire variable set $\mathcal{X}$, fault select variable set $\mathcal{X}_{sel} \subset \mathcal{X}$.

Ensure: Accumulated global fault-enhanced leakage probability ϵ_f.

$\mathsf{f} \leftarrow \mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}$

Function `FaultEnhancedLeakageProbability`$(\mathsf{f}, \mathcal{D}, \mathcal{K})$**:**

- **if** $\mathsf{f} = 0 \vee \mathsf{f} = 1$ **then**
  - **return** f
- **for** $x_i \in \mathcal{X}$ **do**
  - **if** $x_i \notin \mathcal{X}_{sel}$ **then**
    - Select $p_i \in \mathcal{D}$
    - $\epsilon_0 \leftarrow$ `FaultEnhancedLeakageProbability`$(\mathsf{f}|_{x_i=0}, \mathcal{D}\backslash\{p_i\}, \mathcal{K})$
    - $\epsilon_1 \leftarrow$ `FaultEnhancedLeakageProbability`$(\mathsf{f}|_{x_i=1}, \mathcal{D}\backslash\{p_i\}, \mathcal{K})$
    - **return** $\epsilon_0 + p_i \cdot (\epsilon_1 - \epsilon_0)$
  - **else**
    - Select $q_i \in \mathcal{K}$
    - $\epsilon_0 \leftarrow$ `FaultEnhancedLeakageProbability`$(\mathsf{f}|_{x_i=0}, \mathcal{D}, \mathcal{K}\backslash\{q_i\})$
    - $\epsilon_1 \leftarrow$ `FaultEnhancedLeakageProbability`$(\mathsf{f}|_{x_i=1}, \mathcal{D}, \mathcal{K}\backslash\{q_i\})$
    - **return** $\epsilon_0 + q_i \cdot (\epsilon_1 - \epsilon_0)$

Specifically, the FFHT can be computed with linear complexity in the number of MTBDDs variables by leveraging the fundamental MTBDDs operations RESTRICT and ITE.

Leakage Probability Computation from Decomposed Frequency. Finally, we accumulate the frequency decomposition to obtain the actual leakage function $\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}(\mathcal{D}, \mathcal{K})$, as described in Equation 1. To compute the overall leakage probability of the fault-enhanced circuit $\mathsf{C}^{\mathcal{F}}$, we utilize the leakage function $\mathsf{L}_{(\mathsf{Enc},\mathsf{A}_{\mathsf{C}^{\mathcal{F}}})}$ in a modified SATCOUNT algorithm, which accumulates the individual leakage probabilities for each wire by traversing the paths from the root node to the terminal node 1, as outlined in Algorithm 3.

4.2.2 Performance Optimizations.

To mitigate the inherent complexity of the random combined model, we propose and incorporate optimization techniques into our verification framework that aim to reduce the faulting and probing search space, albeit at the cost of slightly reduced accuracy in the verification outcome. These techniques primarily target the minimization of BDD and MTBDD sizes, which is crucial for enhancing the performance of the underlying operations and, in turn, the overall performance of our verification framework. However, it is worth noting that all performance optimizations, which trade off accuracy for speed, can be optionally disabled, allowing users to generate results with maximum accuracy if desired.

Leakage Probability Approximation. Our first optimization strategy involves constraining the number of probe and fault combinations to the subset with at most d probes and k faults, and then randomly sampling a specified number of combinations from this subset. However, this approach inevitably leaves out some probe and fault combinations, resulting in inaccurate leakage probability computations. To account for this, we introduce computation of leakage bounds to approximate the leakage probability: a lower bound

assumes that all omitted combinations do not leak, while an upper bound assumes that all omitted combinations leak for each selected set of k faults. Naturally, the accuracy of this approximation improves as the sample size, d, and k increase, allowing for a controlled tradeoff between performance and accuracy.

Problem Space Partitioning. As a second optimization strategy, we enable the partitioning of the circuit under test into independent components during a preprocessing step that precedes the actual leakage analysis (as shown in Figure 1). This approach aims to reduce the analysis complexity, which typically scales exponentially with the number of wires in the circuit, by dividing it into smaller, manageable parts that can be analyzed separately. To achieve this, we partition the circuit in two dimensions, allowing for a more efficient and scalable analysis process, however, similarly trading performance for accuracy.

Circuit Partitioning in Breadth: For this, we partition the circuit graph into clusters of strongly connected components with respect to the secrets, i. e., specifically targeting components that process independent unshared inputs. This partitioning approach is grounded in the observation that independent components can only leak the secrets that are processed, provided that the processed secret shares are independent. By dividing the circuit into these independent components, we can verify each component separately and then accumulate their analysis results, yielding a significant performance boost *without sacrificing analysis accuracy.*

Circuit Partitioning in Depth: For the depth partitioning, we assume a univariate adversary that can only observe leakage within a single clock cycle. In pipelined or iterative circuits, each combinational circuit stage between registers represents one clock cycle, allowing us to divide the verification process into separate stages. To analyze each stage, we generate the output distribution of the previous stage using the corresponding transition function. We then compute the leakage bounds by accumulating the individually computed leakage functions for each partition. Specifically, for each pipelining stage $\mathsf{C}^{\mathcal{F}}_{\mathcal{S}_i}$ and its subsequent stage $\mathsf{C}^{\mathcal{F}}_{\mathcal{S}_{i+1}}$, we accumulate the FHT-transformed probe frequencies $\mathsf{D}_{\mathsf{H}^{\mathcal{F}}x,i}|\mathcal{P}$, represented as MTBDDs, using the following formula:

$$\mathsf{D}_{\mathsf{H}^{\mathcal{F}}_x}|_{\mathcal{P}} = \bigvee_{i=0}^{|\mathcal{S}-1|} \mathsf{D}_{\mathsf{H}^{\mathcal{F}}_x,i}|_{\mathcal{P}}.$$

By analyzing each cycle iteratively, we can significantly reduce the size of the functions associated with the wires and, thus, the size of consecutive frequency MTBDDs, resulting in a substantial performance increase. However, this approach *sacrifices accuracy* due to the neglect of multivariate information leakage.

5 Evaluation

This section presents our experimental evaluation, comprising (i) a comparative analysis of various gadgets for baseline comparison with VERICA+ and ironMask+, and (ii) an in-depth evaluation of larger constructions highlighting benefits and limitations of our framework.

Benchmarking Environment and Setup. All our experiments were run on a machine equipped with two AMD EPYC 7742 64-Core processors and 512 GB of memory. The results of our experiments are reported for a $(\mathcal{D}, \mathcal{K})$-combined adversary that either injects faults into all combinational or only sequential gates of the circuits. For each experiment, we considered three different fault types (reset, set, flip) and assumed that each fault occurs

Table 2: Comparison of combined analysis tool capabilities in the standard and glitch-extended (robust) probing model. d/k denote verification in the threshold probing/faulting model, and $\mathcal{D}/\mathcal{K}$ denote verification in the random probing/faulting model, respectively. We denote verification for gadgets and larger constructions with ○ and □, respectively, and sampling-based verification with ○ / □, while ● / ■ indicates exhaustive verification.

	Tool	Reference	Circuit	Probing		Faulting		Combined			
			Structure	d	$\mathcal{D}$	k	$\mathcal{K}$	(d,k)	$(\mathcal{D},k)$	$(d,\mathcal{K})$	$(\mathcal{D},\mathcal{K})$
Standard	VERICA+	[BFG+24]	arbitrary	○□	○□	○□	○□	○□	○□	○□	○□
	ironMask+	[BFG+24]	specific	○	○	○	○	○	○	○	○
	INDIANA	[BFG+25]	arbitrary	●□	●□	×	×	×	×	×	×
	— **This work** —		arbitrary	●□	●□	×	×	●□	●□	●□	●□
Robust	VERICA+	[BFG+24]	arbitrary	○□	○□	○□	○□	○□	○□	○□	○□
	ironMask+	[BFG+24]	specific	○	○	○	○	○	○	○	○
	INDIANA	[BFG+25]	arbitrary	●□	×	×	×	×	×	×	×
	— **This work** —		arbitrary	●□	×	×	×	●□	×	●□	×

with probability q, while each wire leaks information with probability p. For simplicity, we assumed that faults and leakages on different wires are independent and that the probabilities are equal across fault and leakage types. Further, changing the leakage and fault probabilities p and q is independent of computing the leakage function and is therefore computationally negligible (i. e., it only requires re-evaluation of the leakage function).

5.1 Comparison to VERICA+ and ironMask+

Among existing approaches, VERICA+ is the tool most comparable to ours (Table 2), and we therefore focus our evaluation on a direct comparison with VERICA+, for which we selected the same set of case studies and conducted experiments in a combined adversary setting. The netlists were obtained from the public repository of [BFG+24][3]. However, although [BFG+24] reports verification results for five circuits, we excluded the DOMREP-II evaluation from our comparison since its non-pipelined design is incompatible with our framework.

1. DOMREP [GPK+21] is a gadget that uses simple replication of the masked DOM-multiplication gadget, as proposed in [GMK17]. This gadget has been enhanced with an output correction module, as described in [BFG+24].
2. SININA [DN20, RBFSG22] is a gadget that was originally proposed for software implementations in [DN20] and later adapted for hardware implementations in [RBFSG22]. It shares similarities with DOMREP, as it also uses replication of the masked DOM-multiplication gadget. However, SININA includes additional corrections after mask refreshing and before compression.
3. HPC_1^C [FRBSG22] is a gadget that introduces additional refresh and correction operations to one of the shared inputs of a replicated DOM-multiplication gadget.
4. CPC_1 [FGM+23] is a variant of HPC_1^C that includes additional corrections after the mask refreshing in the DOM-multiplication gadget.

Observations. Our results, reported in Table 3, show some interesting trends. First, the reported leakage probabilities $\epsilon_{\mathsf{min}}/\epsilon_{\mathsf{max}}$ are accumulated over all possible fault and probe combinations, which corresponds to an adversary unaware of the exact fault. Interestingly, the advantage of a $(\mathcal{D},\mathcal{K})$-combined adversary does not increase, and in some cases even decreases, compared to a random probing adversary of a non-faulty circuit due to destruction of sensitive information. Secondly, the severity of a certain fault type is highly dependent on the targeted circuit and function, as highlighted by the fact that

[3] https://github.com/Chair-for-Security-Engineering/VERICA/tree/verica%2B

Table 3: Verification results for gadgets (w/o input faults), comparing bounded verification with two probes and faults (w/ partitioning) to full verification with all possible probes and faults (w/o partitioning), demonstrating trade-offs between computational complexity and tightness of leakage bounds. All gadgets are protected against one probe and fault. For all experiments, the time out (TO.) was set to 24h.

Scheme	Design					Probing		Faulting		Faults		Leakage		Verif.	Time
	inputs	rand.	comb.	seq.	wires	d	p	k	q	loc.	type	ϵ_{min}	ϵ_{max}	partitions	
DOMREP [GPK+21]	12	1	48	18	120	2	2^{-10}	–	–	–	–	1.113e^{-3}	2.099e^{-1}	3/3	0.5 s
						120	2^{-10}	–	–	–	–	1.908e^{-3}	1.908e^{-3}	1/1	0.2 s
						2	2^{-10}	2	2^{-10}	seq.	set	1.113e^{-3}	2.099e^{-1}	3/3	0.9 s
						120	2^{-10}	19	2^{-10}	seq.	set	1.963e^{-3}	1.963e^{-3}	1/1	27.3 s
						2	2^{-10}	2	2^{-10}	seq.	reset	1.113e^{-3}	2.099e^{-1}	3/3	0.9 s
						120	2^{-10}	19	2^{-10}	seq.	reset	1.963e^{-3}	1.963e^{-3}	1/1	57.8 s
						2	2^{-10}	2	2^{-10}	seq.	flip	1.112e^{-3}	2.099e^{-1}	3/3	0.8 s
						120	2^{-10}	19	2^{-10}	seq.	flip	1.564e^{-3}	1.564e^{-3}	1/1	0.6 s
						2	2^{-10}	2	2^{-10}	comb.	set	1.111e^{-3}	2.099e^{-1}	3/3	6.0 s
						120	2^{-10}	67	2^{-10}	comb.	set	1.961e^{-3}	1.961e^{-3}	1/1	1.2 h
						2	2^{-10}	2	2^{-10}	comb.	reset	1.110e^{-3}	2.099e^{-1}	3/3	2.6 s
						120	2^{-10}	67	2^{-10}	comb.	reset	1.959e^{-3}	1.959e^{-3}	1/1	9.9 min
						2	2^{-10}	2	2^{-10}	comb.	flip	1.110e^{-3}	2.099e^{-1}	3/3	1.8 s
						120	2^{-10}	67	2^{-10}	comb.	flip	1.679e^{-3}	1.679e^{-3}	1/1	13.0 s
CPC_1 [FGM+23]	12	2	78	18	180	2	2^{-10}	–	–	–	–	1.542e^{-3}	2.976e^{-1}	3/3	0.7 s
						180	2^{-10}	–	–	–	–	2.852e^{-3}	2.852e^{-3}	1/1	0.2 s
						2	2^{-10}	2	2^{-10}	seq.	set	1.542e^{-3}	2.976e^{-1}	3/3	1.0 s
						180	2^{-10}	20	2^{-10}	seq.	set	2.860e^{-3}	2.860e^{-3}	1/1	50.8 s
						2	2^{-10}	2	2^{-10}	seq.	reset	1.542e^{-3}	2.976e^{-1}	3/3	1.0 s
						180	2^{-10}	20	2^{-10}	seq.	reset	2.859e^{-3}	2.859e^{-3}	1/1	7.6 s
						2	2^{-10}	2	2^{-10}	seq.	flip	1.541e^{-3}	2.976e^{-1}	3/3	0.9 s
						180	2^{-10}	20	2^{-10}	seq.	flip	2.759e^{-3}	2.759e^{-3}	1/1	1.9 min
						2	2^{-10}	2	2^{-10}	comb.	set	1.537e^{-3}	2.976e^{-1}	3/3	9.0 s
						180	2^{-10}	98	2^{-10}	comb.	set	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	comb.	reset	1.537e^{-3}	2.976e^{-1}	3/3	8.2 s
						180	2^{-10}	98	2^{-10}	comb.	reset	2.852e^{-3}	2.852e^{-3}	1/1	25.7 min
						2	2^{-10}	2	2^{-10}	comb.	flip	1.538e^{-3}	2.976e^{-1}	3/3	3.5 s
						180	2^{-10}	98	2^{-10}	comb.	flip	×	×	0/1	TO.
HPC_1^C [FRBSG22]	12	2	78	24	186	2	2^{-10}	–	–	–	–	2.836e^{-3}	3.067e^{-1}	3/3	0.8 s
						186	2^{-10}	–	–	–	–	4.538e^{-3}	4.538e^{-3}	1/1	0.2 s
						2	2^{-10}	2	2^{-10}	seq.	set	2.836e^{-3}	3.067e^{-1}	3/3	0.9 s
						186	2^{-10}	26	2^{-10}	seq.	set	4.543e^{-3}	4.543e^{-3}	1/1	3.6 s
						2	2^{-10}	2	2^{-10}	seq.	reset	2.836e^{-3}	3.067e^{-1}	3/3	0.9 s
						186	2^{-10}	26	2^{-10}	seq.	reset	4.543e^{-3}	4.543e^{-3}	1/1	4.2 s
						2	2^{-10}	2	2^{-10}	seq.	flip	2.836e^{-3}	3.067e^{-1}	3/3	0.9 s
						186	2^{-10}	26	2^{-10}	seq.	flip	4.447e^{-3}	4.447e^{-3}	1/1	1.5 s
						2	2^{-10}	2	2^{-10}	comb.	set	2.828e^{-3}	3.067e^{-1}	3/3	2.8 s
						186	2^{-10}	104	2^{-10}	comb.	set	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	comb.	reset	2.828e^{-3}	3.067e^{-1}	3/3	2.8 s
						186	2^{-10}	104	2^{-10}	comb.	reset	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	comb.	flip	2.831e^{-3}	3.067e^{-1}	3/3	1.9 s
						186	2^{-10}	104	2^{-10}	comb.	flip	×	×	0/1	TO.
SININA [DN20, RBFSG22]	12	2	90	30	204	2	2^{-10}	–	–	–	–	1.827e^{-3}	4.511e^{-1}	4/4	1.1 s
						204	2^{-10}	–	–	–	–	4.102e^{-3}	4.102e^{-3}	1/1	0.6 s
						2	2^{-10}	2	2^{-10}	seq.	set	1.826e^{-3}	4.511e^{-1}	4/4	1.4 s
						204	2^{-10}	32	2^{-10}	seq.	set	4.099e^{-3}	4.099e^{-3}	1/1	6.9 s
						2	2^{-10}	2	2^{-10}	seq.	reset	1.825e^{-3}	4.511e^{-1}	4/4	1.4 s
						204	2^{-10}	32	2^{-10}	seq.	reset	4.098e^{-3}	4.098e^{-3}	1/1	12.8 s
						2	2^{-10}	2	2^{-10}	seq.	flip	1.826e^{-3}	4.511e^{-1}	4/4	1.5 s
						204	2^{-10}	32	2^{-10}	seq.	flip	2.661e^{-3}	2.661e^{-3}	1/1	6.2 s
						2	2^{-10}	2	2^{-10}	comb.	set	1.823e^{-3}	4.511e^{-1}	4/4	4.5 s
						204	2^{-10}	122	2^{-10}	comb.	set	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	comb.	reset	1.822e^{-3}	4.511e^{-1}	4/4	19.2 s
						204	2^{-10}	122	2^{-10}	comb.	reset	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	comb.	flip	1.825e^{-3}	4.511e^{-1}	4/4	3.2 s
						204	2^{-10}	122	2^{-10}	comb.	flip	×	×	0/1	TO.

leakage probabilities do not consistently decrease across the different experiments. Thirdly, while our approach demonstrates significant improvements, its computational limits are highlighted by the fact that most experiments with combinational faults timed out (24-hour threshold) when considering all possible probe/fault combinations. Table 4 provides further insights into complexity growth for intermediate values[4]. However, for the smallest case study (DOMREP), we achieved a remarkable result, symbolically simulating 2^{67} faulty circuits and exactly computing the probability distribution over 2^{120} different observable states, given 2^{13} input assignments, which underscores the benefits and advancements of our method.

Discussion. While a direct comparison of leakage probabilities with VERICA+ [BFG+24] is not feasible due to differences in fault and leakage considerations, our results demonstrate competitive performance. Notably, our approach outperforms VERICA+ in terms of computation time, with a 20× speedup for the smallest design (DOMREP) and a $2-3$ order of magnitude improvement for larger designs.

Precisely, the complexity of hardware security verification in the (general) random combined model is governed by two main aspects: (i) the complexity of the circuit structure and its underlying Boolean function, and (ii) the complexity of the adversary model. The former aspect, which includes the number of variables of the underlying Boolean functions and their properties, mainly determines whether the BDDs for the circuit can be constructed efficiently and applies as a limitation to both VERICA+ and our tool, although our optimizations (Section 4.2) can partially address this limitation.

In contrast, the complexity of the adversary model, which includes the number of probe and fault combinations to be considered, poses a significant limitation for VERICA+. The explicit generation of all possible combinations by VERICA+ becomes infeasible for large circuits, whereas our approach leverages symbolic and parallel simulation to overcome this limitation. By constructing a fault-enhanced leakage function, we abstract away the underlying circuit structure and functionality and focus solely on the leakage behavior. Although the complexity of this leakage function still depends on the number of probes and faults, our approach enables the computation of exact leakage probabilities for larger probe and fault combinations, whereas VERICA+ fails to do so.

5.2 Extended Verification of Gadgets and Verification Timeouts

Table 4 provides a more comprehensive understanding of the verification runtime behavior with respect to probe/fault set sizes, sample counts for d and k, circuit size and structure, fault models, and partitioning in depth and breadth. For this, we have conducted carefully selected additional evaluations on the gadgets of Table 3 with intermediate parameter settings, particularly in cases where timeouts were encountered previously.

Discussion. As can be seen from the experiments, the verification complexity of our proposed methodology is governed by a close and complex interaction of several parameters.

First, the combinatorial complexity of the underlying probing and faulting adversary models follows a Binomial distribution. This fact is further amplified in the combined context, where the product of the two individual distributions must be taken into account.

Secondly, the size and structure of the underlying Decision Diagrams (DDs) significantly impact the verification performance. The memory and runtime required to process operations on DDs are particularly influenced by the number and ordering of variables, which in turn immediately impacts the complexity of the FHT computation. Thus, large variable counts and/or unfavorable orderings can render operations and FHT computations inefficient, particularly for complex circuits.

[4]We are omitting input faults at this point (similar to VERICA+) only to improve comparability.

Table 4: Extended verification results for timed out gadget verification runs, using intermediate values for different parameters, i.e., with and without partitioning and varying quantities of probes and faults (for all circuits and fault types that timed out). For all experiments, the time out (TO.) was set to 24h.

Scheme	Design					Probing		Faulting		Faults		Leakage		Verif.	Time
	inputs	rand.	comb.	seq.	wires	d	p	k	q	loc.	type	$\epsilon_{\min}$	$\epsilon_{\max}$	partitions	
CPC_1 [FGM+23]	12	2	78	18	180	3	2^{-10}	3	2^{-10}	comb.	set	$1.547e^{-3}$	$2.976e^{-3}$	3/3	30.8 min
						180	2^{-10}	98	2^{-10}	comb.	set	$1.547e^{-3}$	$2.976e^{-3}$	3/3	13.6 h
						2	2^{-10}	2	2^{-10}	comb.	set	$2.818e^{-3}$	$3.222e^{-3}$	1/1	6.9 min
						3	2^{-10}	3	2^{-10}	comb.	set	$2.855e^{-3}$	$2.864e^{-3}$	1/1	13.0 h
						3	2^{-10}	3	2^{-10}	comb.	flip	$1.540e^{-3}$	$2.976e^{-3}$	3/3	27.7 min
						180	2^{-10}	98	2^{-10}	comb.	flip	×	×	0/3	TO.
						2	2^{-10}	2	2^{-10}	comb.	flip	$2.811e^{-3}$	$3.215e^{-3}$	1/1	2.7 min
						3	2^{-10}	3	2^{-10}	comb.	flip	$2.847e^{-3}$	$2.856e^{-3}$	1/1	8.9 h
HPC_1^C [FRBSG22]	12	2	78	24	186	3	2^{-10}	3	2^{-10}	comb.	set	$2.836e^{-3}$	$3.067e^{-1}$	3/3	3.7 min
						186	2^{-10}	104	2^{-10}	comb.	set	$2.836e^{-3}$	$3.067e^{-1}$	3/3	1.1 h
						2	2^{-10}	2	2^{-10}	comb.	set	$4.502e^{-3}$	$4.842e^{-3}$	1/1	1.8 min
						3	2^{-10}	3	2^{-10}	comb.	set	$4.531e^{-3}$	$4.539e^{-3}$	1/1	1.4 h
						3	2^{-10}	3	2^{-10}	comb.	reset	$2.835e^{-3}$	$3.067e^{-1}$	3/3	3.4 min
						186	2^{-10}	104	2^{-10}	comb.	reset	$2.836e^{-3}$	$3.067e^{-1}$	3/3	5.2 min
						2	2^{-10}	2	2^{-10}	comb.	reset	$4.498e^{-3}$	$4.839e^{-3}$	1/1	4.0 min
						3	2^{-10}	3	2^{-10}	comb.	reset	$4.527e^{-3}$	$4.534e^{-3}$	1/1	4.5 h
						3	2^{-10}	3	2^{-10}	comb.	flip	$2.833e^{-3}$	$3.067e^{-1}$	3/3	4.5 min
						186	2^{-10}	104	2^{-10}	comb.	flip	×	×	1/3	TO.
						2	2^{-10}	2	2^{-10}	comb.	flip	$4.499e^{-3}$	$4.840e^{-3}$	1/1	1.7 min
						3	2^{-10}	3	2^{-10}	comb.	flip	$4.529e^{-3}$	$4.536e^{-3}$	1/1	10.8 h
SINI NA [DN20, RBFSG22]	12	2	90	30	204	3	2^{-10}	3	2^{-10}	comb.	set	$1.821e^{-3}$	$4.511e^{-1}$	4/4	1.8 h
						204	2^{-10}	122	2^{-10}	comb.	set	×	×	0/4	TO.
						2	2^{-10}	2	2^{-10}	comb.	set	$4.040e^{-3}$	$4.575e^{-3}$	1/1	6.2 min
						3	2^{-10}	3	2^{-10}	comb.	set	$4.087e^{-3}$	$4.100e^{-3}$	1/1	5.7 h
						3	2^{-10}	3	2^{-10}	comb.	reset	$1.823e^{-3}$	$4.511e^{-1}$	4/4	1.6 h
						204	2^{-10}	122	2^{-10}	comb.	reset	×	×	1/4	TO.
						2	2^{-10}	2	2^{-10}	comb.	reset	$4.042e^{-3}$	$4.577e^{-3}$	1/1	7.4 min
						3	2^{-10}	3	2^{-10}	comb.	reset	$4.089e^{-3}$	$4.102e^{-3}$	1/1	12.1 h
						3	2^{-10}	3	2^{-10}	comb.	flip	$1.824e^{-3}$	$4.511e^{-1}$	4/4	5.2 h
						204	2^{-10}	122	2^{-10}	comb.	flip	×	×	0/4	TO.
						2	2^{-10}	2	2^{-10}	comb.	flip	$4.045e^{-3}$	$4.579e^{-3}$	1/1	9.5 min
						3	2^{-10}	3	2^{-10}	comb.	flip	×	×	0/1	TO.

Thirdly, the circuit size and structure, along with the fault model, also play pivotal roles in determining the verification complexity. Different fault models have varying impacts on the resulting DDs. Likewise, the circuit size directly affects the number of probe and fault locations, i. e., the combinatorial complexity of the adversary models.

Due to the strong interdependence of the parameters, it is evident that determining verification complexity as a function of these parameters is not tractable in a generic way. We even may observe counterintuitive results, where more complex operations on DDs but fewer combinations of probes/faults prove to be more efficient than smaller operations with a larger number of combinations. Hence, for less complex circuits and/or fault models, verifying all probes at once can be more efficient than exhaustively sampling smaller combinations, owing to the potentially fast DD operations and FHT computation in this context. Conversely, for complex circuits, full verification often becomes infeasible, and estimating leakage via sampling smaller probe combinations (on partitioned circuits[5]) is comparatively more efficient.

[5]Performance optimizations such as partitioning and dynamic variable reordering can partially address the challenges, but never prevent the overall increase in complexity.

Table 5: Bounded verification results (w/ input faults) for larger constructions with two probes and faults. All constructions are protected against one probe and fault. For all experiments, the time out (TO.) was set to 24h.

Scheme	Design					Probing		Faulting		Faults		Leakage		Verif.	Time
	inputs	rand.	comb.	seq.	wires	d	p	k	q	loc.	type	$\epsilon_{\min}$	$\epsilon_{\max}$	partitions	
S-box PRESENT-HPC$_1$ [BLH+25]	8	8	82	72	240	2	2^{-10}	–	–	–	–	7.065e^{-4}	2.328e^{-3}	1/1	30.8 s
						2	2^{-10}	2	2^{-10}	seq.	set	7.911e^{-4}	2.411e^{-3}	1/1	2.4 min
						2	2^{-10}	2	2^{-10}	seq.	reset	7.909e^{-4}	2.411e^{-3}	1/1	2.0 min
						2	2^{-10}	2	2^{-10}	seq.	flip	7.040e^{-4}	2.326e^{-3}	1/1	1.3 min
						2	2^{-10}	2	2^{-10}	comb.	set	7.762e^{-4}	2.398e^{-3}	1/1	12.3 min
						2	2^{-10}	2	2^{-10}	comb.	reset	7.759e^{-4}	2.398e^{-3}	1/1	19.2 min
						2	2^{-10}	2	2^{-10}	comb.	flip	6.976e^{-4}	2.321e^{-3}	1/1	2.3 min
Round PRESENT-HPC$_1$ [BLH+25]	256	128	1360	1152	3984	2	2^{-10}	–	–	–	–	4.501e^{-3}	1.000	160/160	1.8 min
						2	2^{-10}	2	2^{-10}	seq.	set	4.280e^{-3}	1.000	160/160	8.8 min
						2	2^{-10}	2	2^{-10}	seq.	reset	4.280e^{-3}	1.000	160/160	9.6 min
						2	2^{-10}	2	2^{-10}	seq.	flip	4.280e^{-3}	1.000	160/160	11.3 min
						2	2^{-10}	2	2^{-10}	comb.	set	4.063e^{-3}	1.000	160/160	18.7 min
						2	2^{-10}	2	2^{-10}	comb.	reset	4.063e^{-3}	1.000	160/160	30.9 min
						2	2^{-10}	2	2^{-10}	comb.	flip	4.063e^{-3}	1.000	160/160	22.3 min
S-box ASCON-HPC$_1$ [BLH+25]	10	10	78	60	220	2	2^{-10}	–	–	–	–	7.925e^{-4}	2.025e^{-3}	1/1	20.4 s
						2	2^{-10}	2	2^{-10}	seq.	set	8.472e^{-4}	2.079e^{-3}	1/1	1.8 min
						2	2^{-10}	2	2^{-10}	seq.	reset	8.467e^{-4}	2.079e^{-3}	1/1	1.9 min
						2	2^{-10}	2	2^{-10}	seq.	flip	7.902e^{-4}	2.024e^{-3}	1/1	48.7 s
						2	2^{-10}	2	2^{-10}	comb.	set	8.354e^{-4}	2.069e^{-3}	1/1	3.1 min
						2	2^{-10}	2	2^{-10}	comb.	reset	8.346e^{-4}	2.068e^{-3}	1/1	3.0 min
						2	2^{-10}	2	2^{-10}	comb.	flip	7.840e^{-4}	2.018e^{-3}	1/1	1.4 min
Round ASCON-HPC$_1$ [BLH+25]	640	640	6272	4480	17280	2	2^{-10}	–	–	–	–	×	×	256/257	TO.
						2	2^{-10}	2	2^{-10}	seq.	set	×	×	256/257	TO.
						2	2^{-10}	2	2^{-10}	seq.	reset	×	×	256/257	TO.
						2	2^{-10}	2	2^{-10}	seq.	flip	×	×	256/257	TO.
						2	2^{-10}	2	2^{-10}	comb.	set	×	×	256/257	TO.
						2	2^{-10}	2	2^{-10}	comb.	reset	×	×	250/257	TO.
						2	2^{-10}	2	2^{-10}	comb.	flip	×	×	256/257	TO.
S-box AES-HPC$_1$ [BLH+25]	16	72	643	552	1787	2	2^{-10}	–	–	–	–	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	seq.	set	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	seq.	reset	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	seq.	flip	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	comb.	set	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	comb.	reset	×	×	0/1	TO.
						2	2^{-10}	2	2^{-10}	comb.	flip	×	×	0/1	TO.

5.3 Verification of Larger Constructions

To further demonstrate the capabilities of our approach, we expand our analysis to encompass more complex circuit structures. Given the scarcity of pipelined and protected hardware designs, we utilize the HADES framework [BLH+25][6] to construct a range of masked S-boxes and cipher rounds. Specifically, we design and implement a masked PRESENT S-box and round (including key addition), a masked ASCON S-box and permutation, as well as a masked AES S-box, leveraging the HPC$_1$ gadget [CGLS21]. This allows us to apply our approach to a diverse set of complex circuit structures, showcasing its versatility for evaluating the security of a wide range of masked hardware designs.

Discussion. The evaluation results for these constructions are summarized in Table 5. A key distinction between this evaluation and our previous assessment of gadgets is the inclusion of input faults, which is essential for evaluating the stand-alone security of these

[6] https://github.com/Chair-for-Security-Engineering/HADES

constructions. Our results show that we were able to successfully verify the security of both the PRESENT and ASCON S-boxes, whereas the AES S-box proved to be too complex for our verification tool. Furthermore, we achieved notable success in verifying the PRESENT round, with all experiments completing in under one hour. In contrast, the verification of the ASCON permutation was only partially successful, with the final partition – which corresponds to the last permutation after the S-box layer – timing out.

The scope of our extended analysis encompasses a wide range of fault scenarios, with the number of considered faults varying from 80 (input and sequential faults on the $\mathsf{ASCON\text{-}HPC}_1$ S-box) to a substantial 12032 faults (input and combinational faults on the $\mathsf{ASCON\text{-}HPC}_1$ permutation). Notably, although our verification process only exhaustively checks all possible combinations of up to 2 faults, we compute the upper and lower bounds on the leakage probabilities considering all possible faults and probes. This leads to the construction of complex fault-enhanced leakage functions, which is only possible due to our partitioning optimizations as it involves a significant number of probe and fault variables, ranging from at least 300 variables for the leakage function of the $\mathsf{ASCON\text{-}HPC}_1$ S-box to 29312 variables for the $\mathsf{ASCON\text{-}HPC}_1$ permutation. This said, the scale and complexity of these leakage functions clearly underscore the challenges of verifying the security of large and complex cryptographic circuits, while also demonstrating the limitations of our approach. Still, we would like to note that we are the first to evaluate such large-scale constructions in the (general) random combined security model.

Further, the tightness of leakage bounds is inversely proportional to the number of unconsidered probe combinations; as the latter increases, the former decreases. For instance, in the context of the PRESENT S-box, partitioning has no effect (Table 5), thereby minimizing the quantity of unconsidered probe combinations. However, when considering the full PRESENT round, with its 160 partitions, all cross-partition probe combinations remain unconsidered, leading to a conservative over-approximation of the upper (resp. lower) bound, where these combinations are assumed to (not) leak.

6 Conclusion

In conclusion, this work introduces a novel methodology for verifying the security of hard ware implementations in the context of CA. Our approach revolves around the derivation of a *fault-enhanced leakage function*, enabling the efficient and precise computation of leakage probabilities in the random probing model with random faults. We demonstrate the practicality of our approach by extending the concepts of *indistinguishability analysis* and the *FHT* to account for random faults. As a result, our work presents the first large-scale verification framework for arbitrary hardware circuits in the random probing model under random faults, as validated by our case studies.

However, we have to note additional limitations of our methodology, particularly when it comes to fault-only security, which requires not only knowledge of the output distribution but also the exact location of output faults, including the order of outputs. Additionally, the random probing and faulting model pose inherent challenges, such as determining practical probe and fault distributions. Still, these challenges and limitations provide opportunities for future research and development, highlighting the need for continued advancements in the field of hardware security verification.

Acknowledgements

The work described in this paper has been supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 510964147 (CAVE), and under Germany's Excellence Strategy – EXC 2092 CASA – 390781972.

References

[ABEO24] Paula Arnold, Sebastian Berndt, Thomas Eisenbarth, and Maximilian Orlt. Polynomial sharings on two secrets: Buy one, get one free. *IACR TCHES*, 2024(3):671–706, 2024.

[AIS18] Prabhanjan Ananth, Yuval Ishai, and Amit Sahai. Private circuits: A modular approach. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 427–455. Springer, Cham, August 2018.

[ANR18] Victor Arribas, Svetla Nikova, and Vincent Rijmen. VerMI: Verification Tool for Masked Implementations. In *25th IEEE International Conference on Electronics, Circuits and Systems, ICECS 2018, Bordeaux, France, December 9-12, 2018*, pages 381–384. IEEE, 2018.

[APZ21] Melissa Azouaoui, Kostas Papagiannopoulos, and Dominik Zürner. Blind Side-Channel SIFA. In *Design, Automation & Test in Europe Conference & Exhibition, DATE 2021, Grenoble, France, February 1-5, 2021*, pages 555–560. IEEE, 2021.

[AVFM07] Frédéric Amiel, Karine Villegas, Benoit Feix, and Louis Marcel. Passive and Active Combined Attacks: Combining Fault Attacks and Side Channel Analysis. In Luca Breveglieri, Shay Gueron, Israel Koren, David Naccache, and Jean-Pierre Seifert, editors, *Fourth International Workshop on Fault Diagnosis and Tolerance in Cryptography, 2007, FDTC 2007: Vienna, Austria, 10 September 2007*, pages 92–102. IEEE Computer Society, 2007.

[AWMN20] Victor Arribas, Felix Wegener, Amir Moradi, and Svetla Nikova. Cryptographic Fault Diagnosis using VerFI. In *2020 IEEE International Symposium on Hardware Oriented Security and Trust, HOST 2020, San Jose, CA, USA, December 7-11, 2020*, pages 229–240. IEEE, 2020.

[BBC+19] Gilles Barthe, Sonia Belaïd, Gaëtan Cassiers, Pierre-Alain Fouque, Benjamin Grégoire, and François-Xavier Standaert. maskVerif: Automated verification of higher-order masking in presence of physical defaults. In Kazue Sako, Steve Schneider, and Peter Y. A. Ryan, editors, *ESORICS 2019, Part I*, volume 11735 of *LNCS*, pages 300–318. Springer, Cham, September 2019.

[BCP+20] Sonia Belaïd, Jean-Sébastien Coron, Emmanuel Prouff, Matthieu Rivain, and Abdul Rahman Taleb. Random probing security: Verification, composition, expansion and new constructions. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 339–368. Springer, Cham, August 2020.

[BDM+20] Sonia Belaïd, Pierre-Évariste Dagand, Darius Mercadier, Matthieu Rivain, and Raphaël Wintersdorff. Tornado: Automatic generation of probing-secure masked bitsliced implementations. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 311–341. Springer, Cham, May 2020.

[BEF+23] Sebastian Berndt, Thomas Eisenbarth, Sebastian Faust, Marc Gourjon, Maximilian Orlt, and Okan Seker. Combined fault and leakage resilience: Composability, constructions and compiler. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 377–409. Springer, Cham, August 2023.

[BFG+24] Sonia Belaïd, Jakob Feldtkeller, Tim Güneysu, Anna Guinet, Jan Richter-Brockmann, Matthieu Rivain, Pascal Sasdrich, and Abdul Rahman Taleb. Formal definition and verification for combined random fault and random probing security. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part VII*, volume 15490 of *LNCS*, pages 167–200. Springer, Singapore, December 2024.

[BFG+25] Christof Beierle, Jakob Feldtkeller, Anna Guinet, Tim Güneysu, Gregor Leander, Jan Richter-Brockmann, and Pascal Sasdrich. INDIANA - verifying (random) probing security through indistinguishability analysis. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 33–63. Springer, Cham, May 2025.

[BGI+18] Roderick Bloem, Hannes Groß, Rinat Iusupov, Bettina Könighofer, Stefan Mangard, and Johannes Winter. Formal verification of masked hardware implementations in the presence of glitches. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 321–353. Springer, Cham, April / May 2018.

[BGR18] Sonia Belaïd, Dahmun Goudarzi, and Matthieu Rivain. Tight private circuits: Achieving probing security with the least refreshing. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 343–372. Springer, Cham, December 2018.

[BLH+25] Fabian Buschkowski, Georg Land, Niklas Höher, Jan Richter-Brockmann, Pascal Sasdrich, and Tim Güneysu. HADES: automated hardware design exploration for cryptographic primitives. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(4):1–45, 2025.

[BMRT22] Sonia Belaïd, Darius Mercadier, Matthieu Rivain, and Abdul Rahman Taleb. IronMask: Versatile verification of masking security. In *2022 IEEE Symposium on Security and Privacy*, pages 142–160. IEEE Computer Society Press, May 2022.

[CDM+25] Gaëtan Cassiers, Siemen Dhooghe, Thorben Moos, Sayandeep Saha, and François-Xavier Standaert. Fly Away: Lifting Fault Security through Canaries and the Uniform Random Fault Model. *IACR Cryptol. ePrint Arch.*, page 863, 2025.

[CFGR10] Christophe Clavier, Benoit Feix, Georges Gagnerot, and Mylène Roussellet. Passive and Active Combined Attacks on AES - Combining Fault Attacks and Side Channel Analysis. In Luca Breveglieri, Marc Joye, Israel Koren, David Naccache, and Ingrid Verbauwhede, editors, *2010 Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTC 2010, Santa Barbara, California, USA, 21 August 2010*, pages 10–19. IEEE Computer Society, 2010.

[CFOS21] Gaëtan Cassiers, Sebastian Faust, Maximilian Orlt, and François-Xavier Standaert. Towards tight random probing security. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 185–214, Virtual Event, August 2021. Springer, Cham.

[CGLS21] Gaëtan Cassiers, Benjamin Grégoire, Itamar Levi, and François-Xavier Standaert. Hardware Private Circuits: From Trivial Composition to Full Verification. *IEEE Trans. Computers*, 70(10):1677–1690, 2021.

[CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 398–412. Springer, Berlin, Heidelberg, August 1999.

[DAN+18] Lauren De Meyer, Victor Arribas, Svetla Nikova, Ventzislav Nikov, and Vincent Rijmen. M&M: Masks and macs against physical attacks. *IACR TCHES*, 2019(1):25–50, 2018.

[DDF14] Alexandre Duc, Stefan Dziembowski, and Sebastian Faust. Unifying leakage models: From probing attacks to noisy leakage. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 423–440. Springer, Berlin, Heidelberg, May 2014.

[DDRT12] Amine Dehbaoui, Jean-Max Dutertre, Bruno Robisson, and Assia Tria. Electromagnetic Transient Faults Injection on a Hardware and a Software Implementations of AES. In Guido Bertoni and Benedikt Gierlichs, editors, *2012 Workshop on Fault Diagnosis and Tolerance in Cryptography, Leuven, Belgium, September 9, 2012*, pages 7–15. IEEE Computer Society, 2012.

[DEG+18] Christoph Dobraunig, Maria Eichlseder, Hannes Groß, Stefan Mangard, Florian Mendel, and Robert Primas. Statistical ineffective fault attacks on masked AES with fault countermeasures. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 315–342. Springer, Cham, December 2018.

[DLM19] Mathieu Dumont, Mathieu Lisart, and Philippe Maurine. Electromagnetic Fault Injection : How Faults Occur. In *2019 Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTC 2019, Atlanta, GA, USA, August 24, 2019*, pages 9–16. IEEE, 2019.

[DN20] Siemen Dhooghe and Svetla Nikova. My gadget just cares for me - how NINA can prove security against combined attacks. In Stanislaw Jarecki, editor, *CT-RSA 2020*, volume 12006 of *LNCS*, pages 35–55. Springer, Cham, February 2020.

[DN23] Siemen Dhooghe and Svetla Nikova. The Random Fault Model. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *Selected Areas in Cryptography - SAC 2023 - 30th International Conference, Fredericton, Canada, August 14-18, 2023, Revised Selected Papers*, volume 14201 of *Lecture Notes in Computer Science*, pages 191–212. Springer, 2023.

[DN24] Siemen Dhooghe and Svetla Nikova. The random fault model. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *SAC 2023*, volume 14201 of *LNCS*, pages 191–212. Springer, Cham, August 2024.

[DOT24] Siemen Dhooghe, Artemii Ovchinnikov, and Dilara Toprakhisar. StaTI: Protecting against fault attacks using stable threshold implementations. *IACR TCHES*, 2024(1):229–263, 2024.

[FGM+18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and François-Xavier Standaert. Composable masking schemes in the presence of physical defaults & the robust probing model. *IACR TCHES*, 2018(3):89–120, 2018.

[FGM+23] Jakob Feldtkeller, Tim Güneysu, Thorben Moos, Jan Richter-Brockmann, Sayandeep Saha, Pascal Sasdrich, and François-Xavier Standaert. Combined private circuits - combined security refurbished. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 990–1004. ACM Press, November 2023.

[FOWZ25] Sebastian Faust, Maximilian Orlt, Kathrin Wirschem, and Liang Zhao. All-you-can-compute: Packed secret sharing for combined resilience. *IACR TCHES*, 2025(2):420–459, 2025.

[FRBSG22] Jakob Feldtkeller, Jan Richter-Brockmann, Pascal Sasdrich, and Tim Güneysu. CINI MINIS: Domain isolation for fault and combined security. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 1023–1036. ACM Press, November 2022.

[GHP+21] Barbara Gigerl, Vedad Hadzic, Robert Primas, Stefan Mangard, and Roderick Bloem. Coco: Co-design and co-verification of masked software implementations on CPUs. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 1469–1468. USENIX Association, August 2021.

[GMK17] Hannes Groß, Stefan Mangard, and Thomas Korak. An efficient side-channel protected AES implementation with arbitrary protection order. In Helena Handschuh, editor, *CT-RSA 2017*, volume 10159 of *LNCS*, pages 95–112. Springer, Cham, February 2017.

[GMO01] Karine Gandolfi, Christophe Mourtel, and Francis Olivier. Electromagnetic analysis: Concrete results. In Çetin Kaya Koç, David Naccache, and Christof Paar, editors, *CHES 2001*, volume 2162 of *LNCS*, pages 251–261. Springer, Berlin, Heidelberg, May 2001.

[GPK+21] Michael Gruber, Matthias Probst, Patrick Karl, Thomas Schamberger, Lars Tebelmann, Michael Tempelmeier, and Georg Sigl. DOMREP-An Orthogonal Countermeasure for Arbitrary Order Side-Channel and Fault Attack Protection. *IEEE Trans. Inf. Forensics Secur.*, 16:4321–4335, 2021.

[HB21] Vedad Hadzic and Roderick Bloem. COCOALMA: A versatile masking verifier. In *Formal Methods in Computer Aided Design, FMCAD 2021, New Haven, CT, USA, October 19-22, 2021*, pages 1–10. IEEE, 2021.

[HCP+24] Vedad Hadzic, Gaëtan Cassiers, Robert Primas, Stefan Mangard, and Roderick Bloem. Quantile: Quantifying information leakage. *IACR TCHES*, 2024(1):433–456, 2024.

[IPSW06] Yuval Ishai, Manoj Prabhakaran, Amit Sahai, and David Wagner. Private circuits II: Keeping secrets in tamperable circuits. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 308–327. Springer, Berlin, Heidelberg, May / June 2006.

[ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 463–481. Springer, Berlin, Heidelberg, August 2003.

[KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 388–397. Springer, Berlin, Heidelberg, August 1999.

[Koc96] Paul C. Kocher. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In Neal Koblitz, editor, *CRYPTO'96*, volume 1109 of *LNCS*, pages 104–113. Springer, Berlin, Heidelberg, August 1996.

[KSM20] David Knichel, Pascal Sasdrich, and Amir Moradi. SILVER - statistical independence and leakage verification. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part I*, volume 12491 of *LNCS*, pages 787–816. Springer, Cham, December 2020.

[MM22] Nicolai Müller and Amir Moradi. PROLEAD A probing-based hardware leakage detection tool. *IACR TCHES*, 2022(4):311–348, 2022.

[MS06] Stefan Mangard and Kai Schramm. Pinpointing the side-channel leakage of masked AES hardware implementations. In Louis Goubin and Mitsuru Matsui, editors, *CHES 2006*, volume 4249 of *LNCS*, pages 76–90. Springer, Berlin, Heidelberg, October 2006.

[NOV+22] Pascal Nasahl, Miguel Osorio, Pirmin Vogel, Michael Schaffner, Timothy Trippel, Dominic Rizzo, and Stefan Mangard. SYNFI: Pre-silicon fault analysis of an open-source secure element. *IACR TCHES*, 2022(4):56–87, 2022.

[PBGS24] Matthias Probst, Manuel Brosch, Michael Gruber, and Georg Sigl. DOMREP II. In *IEEE International Symposium on Hardware Oriented Security and Trust, HOST 2024, Tysons Corner, VA, USA, May 6-9, 2024*, pages 112–121. IEEE, 2024.

[PR13] Emmanuel Prouff and Matthieu Rivain. Masking against side-channel attacks: A formal security proof. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 142–159. Springer, Berlin, Heidelberg, May 2013.

[RBFSG22] Jan Richter-Brockmann, Jakob Feldtkeller, Pascal Sasdrich, and Tim Güneysu. VERICA - verification of combined attacks automated formal verification of security against simultaneous information leakage and tampering. *IACR TCHES*, 2022(4):255–284, 2022.

[RBSS+21] Jan Richter-Brockmann, Aein Rezaei Shahmirzadi, Pascal Sasdrich, Amir Moradi, and Tim Güneysu. FIVER - robust verification of countermeasures against fault injections. *IACR TCHES*, 2021(4):447–473, 2021.

[RG20] Jan Richter-Brockmann and Tim Güneysu. Improved Side-Channel Resistance by Dynamic Fault-Injection Countermeasures. In *31st IEEE International Conference on Application-specific Systems, Architectures and Processors , ASAP 2020, Manchester, United Kingdom, July 6-8, 2020*, pages 117–124. IEEE, 2020.

[RLK11] Thomas Roche, Victor Lomné, and Karim Khalfallah. Combined Fault and Side-Channel Attack on Protected Implementations of AES. In Emmanuel Prouff, editor, *Smart Card Research and Advanced Applications - 10th IFIP WG 8.8/11.2 International Conference, CARDIS 2011, Leuven, Belgium, September 14-16, 2011, Revised Selected Papers*, volume 7079 of *Lecture Notes in Computer Science*, pages 65–83. Springer, 2011.

[RSG23] Jan Richter-Brockmann, Pascal Sasdrich, and Tim Güneysu. Revisiting Fault Adversary Models - Hardware Faults in Theory and Practice. *IEEE Trans. Computers*, 72(2):572–585, 2023.

[SA03] Sergei P. Skorobogatov and Ross J. Anderson. Optical fault induction attacks. In Burton S. Kaliski, Jr., Çetin Kaya Koç, and Christof Paar, editors, *CHES 2002*, volume 2523 of *LNCS*, pages 2–12. Springer, Berlin, Heidelberg, August 2003.

[SBJ+21] Sayandeep Saha, Arnab Bag, Dirmanto Jap, Debdeep Mukhopadhyay, and Shivam Bhasin. Divided we stand, united we fall: Security analysis of some SCA+SIFA countermeasures against SCA-enhanced fault template attacks. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part II*, volume 13091 of *LNCS*, pages 62–94. Springer, Cham, December 2021.

[SJB+18] Sayandeep Saha, Dirmanto Jap, Jakub Breier, Shivam Bhasin, Debdeep Mukhopadhyay, and Pallab Dasgupta. Breaking Redundancy-Based Countermeasures with Random Faults and Power Side Channel. In *2018 Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTC 2018, Amsterdam, The Netherlands, September 13, 2018*, pages 15–22. IEEE Computer Society, 2018.

[SMG16] Tobias Schneider, Amir Moradi, and Tim Güneysu. ParTI – towards combined hardware countermeasures against side-channel and fault-injection attacks. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 302–332. Springer, Berlin, Heidelberg, August 2016.

[SRJB23] Sayandeep Saha, Prasanna Ravi, Dirmanto Jap, and Shivam Bhasin. Non-profiled side-channel assisted fault attack: A case study on DOMREP. In *Design, Automation & Test in Europe Conference & Exhibition, DATE 2023, Antwerp, Belgium, April 17-19, 2023*, pages 1–6. IEEE, 2023.

[TGS+24] Huiyu Tan, Pengfei Gao, Fu Song, Taolue Chen, and Zhilin Wu. SAT-based formal verification of fault injection countermeasures for cryptographic circuits. *IACR TCHES*, 2024(4):1–39, 2024.

[TNN24] Dilara Toprakhisar, Svetla Nikova, and Ventzislav Nikov. SoK: Parameterization of fault adversary models connecting theory and practice. In Elisabeth Oswald, editor, *CT-RSA 2024*, volume 14643 of *LNCS*, pages 433–459. Springer, Cham, May 2024.

[ZDCT13] Loïc Zussa, Jean-Max Dutertre, Jessy Clédière, and Assia Tria. Power supply glitch induced faults on FPGA: An in-depth analysis of the injection mechanism. In *2013 IEEE 19th International On-Line Testing Symposium (IOLTS), Chania, Crete, Greece, July 8-10, 2013*, pages 110–115. IEEE, 2013.